MICROCONTROLLERS

CHAPTER 15

15.1 Introduction

As we remarked in the previous chapter, microcontrollers are essentially stand-alone processors, intended to be *embedded* into some electronic device that is definitely not a "computer."[1] They are inexpensive and easy to use, and they let you put the intelligence of a computer into pretty much anything electronic. In so doing, they trade off some of the terrifying speed of computer-oriented processors in favor of built-in memory and peripherals.

To expand on this point, nearly all microcontrollers (including those that cost less than a dollar) include both data memory (static RAM) and non-volatile program memory ("flash") on-chip; many include additional non-volatile EEPROM for holding calibration data, system configuration settings, and so on.[2] Better still, you can choose among integrated "peripherals": communication links such as SPI, I^2C, USB, Ethernet, Bluetooth, and ZigBee; buses such as PCI and PCIe, SATA, PCMCIA, flash-memory cards, and external memory; analog interfaces such as comparators, multiplexed ADCs, DACs, and video and imaging sensors; and special-purpose interfaces such as pulse-width modulators, LCD drivers, GPS, digital audio, and WiFi.

To put it another way (Figure 15.1), a microcontroller includes within it the "CPU + memory + peripherals" that required an external data bus and separate components in the previous chapter. (You *can* implement an external data bus with a microcontroller, if you want to; but it's better to choose a microcontroller that has what you want on-chip.)

Of course, in an embedded system you don't boot up from a disk – you don't even *have* a disk! That's why the program memory is included on the controller, and also why it must be non-volatile: your dishwasher needs to know what it is supposed to do, even after a power failure. Because microcontrollers are built into the target device (and generally not removable from the circuit board) they must be programmable (and reprogrammable) *in-circuit*. You program them via an "in-circuit serial programming" interface ("ICSP," usually SPI or JTAG), with a hardware interface "pod," controlled by software running on a host computer.

Microcontrollers are fun, and easy. Think of them as a circuit "component," like an op-amp. The analogy is good: just as the op-amp is the universal analog component, the microcontroller might be thought of as the universal digital component.[3] And a microcontroller plus a few other components (e.g., a USB connector, the programming header, a few lights and buttons, and perhaps an alphanumeric LCD) put onto a small circuit board, is a "universal block" whose function is flexibly programmed to do whatever you want. The microcontroller manufacturers happily provide such objects (sometimes called "development kits," which often include programming hardware and software; see Figure 15.24 on page 1090), to encourage you to adopt their wares. There are also third-party and open-source products, such as "Ethernut" and "Arduino".[4] The former includes an Ethernet port; the latter offers serial, USB, SPI, and I^2C ports, with an optional stackable Ethernet adapter.

Let's launch into microcontrollers with a concrete example.

[1] A recent publication from Maxim Integrated Products (App Note 3967) begins with the sentence "The heart of today's advanced electronic products is a microcontroller (μC) that communicates with one or more peripheral devices." As a friendly amendment, we'd probably omit the word "advanced."

[2] *EEPROM* permits rewriting (or erasing) individual stored bytes, as contrasted with *flash* memory, in which erasure (required before rewriting) can be done only on a block of bytes at once. That is why the lower-density EEPROM is better for storing user data, whereas the higher-density flash is best for infrequently rewritten program code. Both memory types are non-volatile and in-circuit programmable, though flash typically has less rewriting "endurance," e.g., 10,000 erase–write cycles for flash, versus 100,000 for EEPROM.

[3] Although experienced users of FPGAs might well argue that *their* chips are "more universal": you can put a microcontroller "soft core" into an FPGA, but not vice versa.

[4] Here are a few more: BeagleBoard & BeagleBone, Odroid, Raspberry Pi, and Teensy.

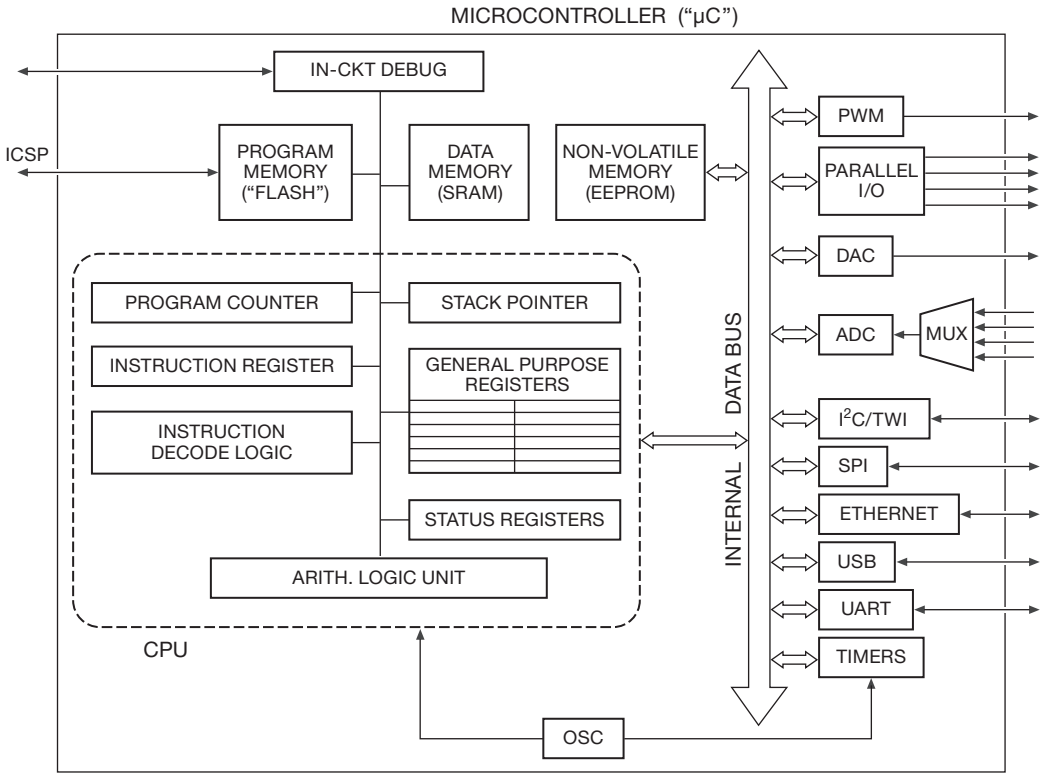


Figure 15.1. A microcontroller integrates memory and "peripherals" onto the same chip as the CPU. ICSP stands for "in-circuit serial programming."

15.2 Design example 1: suntan monitor (V)

We've visited this indispensable device for the beachgoer, first in Chapter 4 (where we explored three purely analog implementations), then again in Chapter 13 (with a photocurrent-integrating ADC). Those illustrated nicely the use of discrete analog and digital electronics. We conclude our pursuit of the perfect suntan integrator in this chapter, with (naturally) a microcontroller implementation. We take a first stab at it here, with a bit of refinement later in the chapter.

To review, the task is to inform the sunbather when to turn over (or go home), after having received the desired accumulated dose of sunlight, using as input a photodiode's current (which is proportional to suntanning intensity). The beachgoer sets the target FSE (full-sunlight-equivalent) dose with a pot, on a scale of 0–90 min FSE, pushes the START button, and begins the bake cycle. A piezo buzzer beeps when the job is complete.

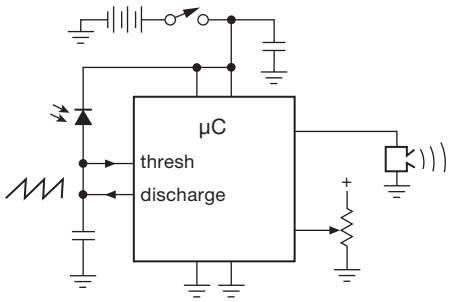


Figure 15.2. Suntan monitor, simply implemented with a microcontroller (μC).

15.2.1 Implementation with a microcontroller

An classic (but uninspired) way to approach this is to convert the current to a voltage (with op-amp and feedback resistor), then use the microcontroller's on-chip ADC to generate digitized samples, which are then integrated numerically. But there's a simpler (and better) way, easily implemented even in a 50-cent microcontroller. Look at Figure 15.2. The idea is to use the photocurrent to ramp the

voltage on a capacitor, then use the microcontroller's analog comparator to trigger a short discharge pulse on a three-state digital output port.[5] This generates a sawtooth whose frequency is proportional to light intensity; the microcontroller integrates this by simply counting cycles, sounding the horn at the desired terminal count, which it derives from a measurement of the "dose" pot's setting. A digital input-port bit senses the START button, and a digital output-port bit activates the piezo buzzer.

Not only is this simpler than the humdrum $I \rightarrow V \rightarrow$ADC method, it is better: converting a current to a proportional voltage requires good dynamic range (and therefore very low offset voltages) to integrate accurately at low light intensities, whereas this $I \rightarrow f$ oscillator preserves accuracy by generating a fixed-amplitude sawtooth with excellent linearity at low currents. This departure from the usual modular engineering design approach illustrates nicely the sort of "holistic" creativity that microcontrollers stimulate in their devotees.

The circuit of Figure 15.2 is by far the simplest of the five iterations of a suntanning circuit, though, of course, there is the task of writing and downloading the embedded code (which we'll look at next). Before we do that, let's flesh out the circuit in all its glory – component values, choice of controller, PINs, etc. There's plenty to learn by doing this; in an important sense, dealing with the details and the compromises they compel is the essence of electronic circuit design.

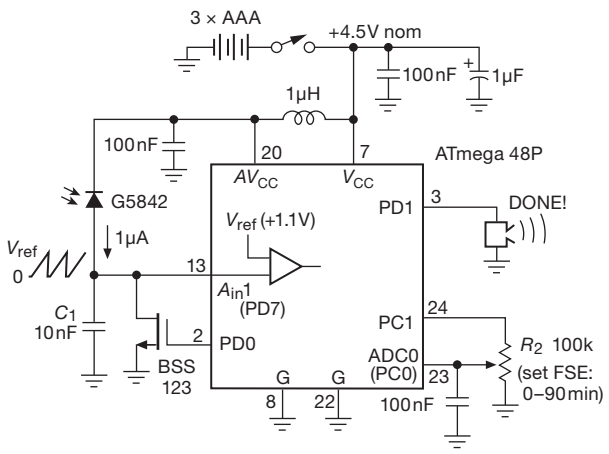


Figure 15.3. Detailed circuit of the suntan monitor. See Figure 4.93 for information about the G5842 GaAsP photosensor.

[5] You could arrange things to do the discharge on the same pin as the threshold comparison, by dynamically changing a bit in the register that sets the port-bit modes.

Figure 15.3 is a complete implementation, using parts available at the time this is being written. Let's look at some details.

A. Choice of microcontroller

This gadget doesn't demand much, in terms of speed or peripherals, and nearly all microcontrollers include a comparator and an ADC. However, we want low current consumption, and we'd like to run directly from a battery, without a voltage regulator. We chose this part because it is the smallest of the "picoPower" series of Atmel's AVR controllers, which have the virtue of low-power operation even when running from +5 V.[6] It operates from 1.8–5.5 V, and draws about 35–170 μA supply current (over that range of supply voltages) when running from its internal 128 kHz oscillator.

B. Discharge circuit

Although you could probably get away with using a three-state output to discharge the oscillator's capacitor directly (Figure 15.2), the worst-case specifications would deny you the satisfaction of a job well done: they specify a maximum leakage current of $\pm 1\,\mu$A, comparable to the photocurrent in sunlight. You could choose a larger photocell, with correspondingly higher photocurrent; but an external *n*-channel MOSFET does the job nicely.[7] Here we've used a small BSS123 *n*-channel MOSFET as an external switch. It's a good choice: it has a satisfyingly low threshold, with $\sim$0.5 A drain current at 2.5 V of gate drive, and a worst-case leakage of 10 nA at V_{DS}=20 V. It's stocked widely, priced at \$0.05 in quantity.[8]

C. Dose-setting potentiometer

The simple diagram biased the pot from the positive supply rail. But good design requires its bias current to be large compared with the 1 μA input leakage spec, which adds significant power drain. We can do better, though, by powering the pot only when we need to, i.e., when reading its value. That's easy – just tie its topside to a digital output-port bit; when the software sets that bit HIGH, the internal *p*-channel pullup brings it to $+V_{CC}$.

[6] Many controllers run at low power when operated at low supply voltage (say 1.8 V), but not at 3–5 V, as here.

[7] You could, instead, be reasonably confident that the typical leakage current is probably less than 10 nA, particularly since the specifications are given over the full temperature range of $-40°$C to $+85°$C; furthermore, semiconductor manufacturers are notoriously conservative when specifying leakage. You don't have to worry about the comparator's input pin, though: its specified leakage current is 50 nA (max).

[8] The BSS123 is similar to the 2N7000 and 2N7002 (see Table 3.4a).

D. Bypassing and decoupling

Figure 15.3 is decorated with decoupling capacitors, perhaps a bit conservatively. We decoupled the analog supply rail (AV_{CC}) from the noisy digital V_{CC} with the *LC* filter recommended in the datasheet. We also bypassed the pot readout, because it is biased from an equally noisy digital output. Finally, we used a parallel combination of supply-rail bypass capacitors to maintain low impedance over frequencies. (The series inductance and resistance of large-value capacitors reduce their effectiveness at high frequencies, remedied here by the smaller 100 nF ceramic bypass capacitor; see Chapter *1x*). Once again, we could probably get away with less, particularly since we need very little accuracy ($\sim$6 bits) from the part's too-good internal 10-bit ADCs, for which the bypass specifications were intended. But we like to play it safe.

E. Oscillator

There isn't one! That's because this controller, in common with many, includes internal oscillator options; for this chip there are two internal oscillators (8 MHz and 128 kHz), of modest accuracy ($\pm$10%).

15.2.2 Microcontroller code ("firmware")

The embedded software, called *firmware*, needs to do several tasks, namely,

(a) initializations at power-up (set up I/O port modes, ADC and comparator modes, and reset state of output port bits);
(b) assert V_{CC} to the pot, wait 25 ms, read its voltage, then disassert;
(c) use the measured voltage to compute the terminal count of oscillation cycles from the sawtooth oscillator, corresponding to the integrated sunlight exposure set by the pot;
(d) use the comparator output state to increment the cycle counter and to generate capacitor discharge pulses;
(e) when the counter reaches the computed terminal count, activate the piezo buzzer.

In addition, there are some operating characteristics (for example, oscillator frequency) that are not under program control and that are set instead at the time the program code is downloaded to the device. More, soon, about programming these so-called "fuses."

A. Pseudocode

We've written this in somewhat more detail in Pseudocode 15.1; pseudocode is a less formal substitute to a graphical *flowchart*, with more readable text.

A bit of explanation: the code begins with some **Setup** tasks. That is because contemporary microcontrollers are richly endowed, with built-in peripherals, operating modes, pin function options, and the like.[9] This flexibility and power are both a blessing and a curse. The blessings are obvious. The curse extracts its revenge, in requiring you to configure all those options. That is done by setting or clearing bits in an array of registers whose function is to configure and control the device's options. This particular processor has 256 such internal registers, with some 40 bytes worth of configuration and control bits. You generally don't have to worry about most of them, because they default to reasonable values; but you do have to set up the range and mode of the ADC and comparator, and the port bit directions.

The program code is executed on power-up, and is obliged to do first this fussy up-front housekeeping: microcontroller port bits can individually be configured as inputs or outputs (under program control), which you specify by setting bits in a port direction register (if an input, you can also enable or disable internal pullup). The setup code continues by initializing the buzzer output bit LOW and the discharge pin HIGH, and concludes with choice of ADC and comparator voltage references, scaling, and input sources. There are a few other configuration bits that have to be set, specifically the clock source (external or internal), the clock frequency, and the clock divider ratio. For this particular microcontroller, these "fuse" bits are set during hardware device programming, rather than in the executable program code.[10]

Next it **Initializes** LOW the two output port bits (beeper, discharge transistor drive), and defines (and clears) a register variable that will hold the accumulating COUNT from the sunlight-proportional sawtooth oscillator. Now we **Read** the pot that sets the target dose: set PC1 to put V_{CC} across the pot, wait 25 ms, start the ADC, hang on its busy flag, then read the high byte of the 10-bit conversion (we really don't need better than 0.5% on this job), power down the pot, and compute the corresponding terminal count. The latter assumes a nominal 100 Hz full-sunlight sawtooth, ($\sim 1\,\mu$A photocurrent), and a full-scale setting of 90 minutes.

All this has taken a fraction of a second. Now we

[9] For example, this processor's pin 28 is bit 5 of byte-wide bidirectional port C; it can also be used as an analog input to the ADC (via the internal 8-input multiplexer), or as the SCL serial clock, or as a "pin-change interrupt" source.

[10] For high-pincount microcontrollers it's common to see a few pins used to determine what happens at boot-up, with program control then in charge of those parameters.

Pseudocode 15.1 Suntan monitor pseudocode

Setup

`Variables:` Define 32-bit integers `count` and `termcount`
`Low Power:` Disable unused peripherals
`Ports:` Set as outputs PC1 (pot bias), PD0 (cap discharge) and PD1 (buzzer);
set as analog inputs PC0 (pot readout) and PD7 (cap voltage);
initialize PD1 LOW (buzzer off), and PD0 HIGH (cap held at gnd)
`Analog modes:` Set comparator mode & bandgap ref;
set ADC ref to Vcc, ADC mode to left-adjusted, and MUX to channel 0

Read "bake" setting

Setbits PC1 (power to pot) and ADEN (ADC enable); wait 100 ms
Start ADC conversion (setbit ADSC)
Wait while ADSC=1 (busy), then read unsigned 8-bit result ("bake")
Clearbits PC1 and ADEN, and disable ADC to save power
Compute `termcount`=360000 x bake/256

Count Cycles

Wait while comparator is HIGH ($V_{cap} < V_{ref}$), then:
setbit PD0, then clearbit PD0 (software discharge pulse)
increment `count`
if `count<termcount`, repeat **Count Cycles**
otherwise set bit PD1 (buzzer), clear 10 sec later

Count cycles of the sawtooth, whose ramp is generated by the photocurrent, abruptly discharged by a software pulse (HIGH, then LOW) to the MOSFET. Because there's nothing else to do anyway, we've used simple polling of the comparator's output bit, rather than the more elegant alternative of a counter-driven interrupt (our comparator mode setup disabled its interrupt). When the count reaches the computed target we turn on the beeper and hang "forever" (that is, until the bakee switches the thing off; restart at your peril – twice-baked human!).

B. Detailed C-code

Program 15.1 is a listing of the C-language source code. It should be readable by fluent C programmers, though there are a few quirks associated with microcontrollers. The `io.h` and `fuse.h` files contain the idiosyncratic register definitions, functions, etc., that deal with peculiarities such as fragmented address spaces and with bit variables. Bit twiddling in ports and registers is common in microcontroller applications; it's done here by OR and AND masking, getting the bit positions from the `defines`, and creating the OR mask with "1<<n" and the AND mask with "~(1<<n)". (For example, `PORTC1` has value 1, as defined in `io.h`; so the program statement "`#define POT (1<<PORTC1)`" sets `POT` equal to 0x02.) And a tricky *gotcha*: in an application like this, in which values are read from memory-mapped ports (such as a comparator or ADC), it is essential to declare those variables as `volatile` to prevent the compiler from "optimizing" away the reload from the associated port. This has been done for you in the `io.h` file, for the standard predefined memory locations; but you'd need to do it for any custom memory-mapped variables you create.

C. Some comments

(a) We used the power-on switch to initiate the timing cycle; another way to handle this is to use a pushbutton that brings an input pin to ground; you could poll that input bit, or you could use a level-sensitive interrupt. The latter is more elegant and has the benefit that the processor could put itself into a micropower ($<1\,\mu$A) "sleep" state when done; the processor would then wake up upon a pin LOW interrupt. In principle you could even omit the power switch! We'd advise against that, though, because the only recovery from a crashed processor would be to remove the battery. We've all experienced commercial electronic devices (answering machines, cameras, DVRs) that occasionally freeze up, requiring a "cold reboot" by removing the battery or unplugging the device.

Program 15.1

```
#include <avr/io.h>
#include <avr/fuse.h>
#include <util/delay.h>

#define DISCHARGE  (1<<PORTD0)
#define BUZZER     (1<<PORTD1)
#define POT        (1<<PORTC1)

int main() {
    long  termcount, count;  // Total timer counts and running counter

    // Power saving measures
    PRR = ~(1<<PRADC) & ~(1<<PRSPI); // Shut off peripherals except ADC & SPI
    DIDR0  = 0x3f;  // Disable digital input buffers on analog pins

    // Setup the pins
    DDRD   = DISCHARGE | BUZZER;  // Set two pins to output, rest to input
    DDRC   = POT;         // Set the POT pin to output, rest to input
    DIDR0 |= (1<<ADC0D);  // Use PC0 as ADC0 -- the ADC input
    DIDR1 |= (1<<AIN1D);  // Use PD7 as AIN1 -- the comparator input
    PORTD  = BUZZER;  // Hold cap low, and start with buzzer off

    // Comparator Setup
    ACSR = (1<<ACBG); // Set the reference to the band gap (needs 70 us)

    // Read the desired exposure duration
    PORTC |= POT;  // Turn on the top of the resistor divider.
    _delay_ms(25);  // Delay 25 ms for 10RC settle
    ADMUX  = (1<<REFS0) | (1<<ADLAR); // Use Vcc ref; left-adjusted result
    ADCSRA = (1<<ADEN) | (1<<ADSC);  // Enable, and start ADC

    /*** Wait until ADC conversion is done. ***/
    while ( ADCSRA & (1<<ADSC) ) { }

    termcount = (360000L * ADCH) >> 8; // Convert ADC result to timer count
    PORTC &= ~POT;  // Turn off the top of the resistor divider
    ADCSRA &= ~(1<<ADEN); // Disable ADC
    PRR |= (1<<PRADC);  // Enable power-reduction for ADC

    /*** Wait until desired sunlight exposure ***/
    for (count = 0; count < termcount; count++) {
        // Wait for cap to charge, then comparator output goes low
        while(ACSR & (1<<ACO)) { }
        PORTD |= DISCHARGE;  // Discharge the capacitor
        PORTD &= ~DISCHARGE; // And release it to recharge
    }

    // Buzz for 10 seconds
    PORTD |= BUZZER;     // Power the buzzer
    _delay_ms(10*1000);  // Delay 10 seconds
    PORTD = BUZZER;      // Turn off buzzer

    // Loop forever
    while (1) { }
}
```

(b) We polled the comparator to determine when the sawtooth reached the trigger voltage; you could, instead, use an interrupt. There's no benefit here, however, because there's nothing else to do anyway: with an interrupt you would discharge the capacitor and increment the counter in the interrupt service routine, while the main program would simply loop on testing the value in the counter.
(c) Microcontrollers are *smart*! With hardly any work we can add all kinds of cool features to this puppy: an LCD display with a "progress bar" to show the state of your tanning; a readout of sunlight intensity; an MP3 music player; a game of solitaire or scrabble…(but we digress: you get the point).

D. Downloading the code

The last step is to "program" the part (i.e., download the compiled code and set the fuses). Here's what you do.
(a) Connect the pod to the appropriate pins on the microcontroller (via a programming header), and to the PC (through serial or USB).
(b) Power up your board.
(c) Use the software on the PC to verify that the pod detects the correct device.
(d) Decide on the fuse-settable options,[11] and program them.
(e) Select the location of the compiled HEX file on the PC, and download it into the microcontroller flash memory; optionally select a HEX file with the EEPROM contents and download it.
(f) The pod should automatically reset the chip and begin running your program. At this point you can remove the pod. When you power cycle your board, your code will run, no strings attached.

E. Human-interface review

Our astute colleague Jim MacArthur offered the following commentary.

> If this book were about human interface (HI) design, design example #1 would be a textbook case of *spec drift*. The specification couldn't have been simpler: Duplicate the functionality of the analog solution. And yet, as your HI tester will doubtless tell you, this solution differs from the analog one in one important way: If you adjust the pot in the middle of the cooking cycle, the analog solution pays attention, whereas the digital one does not.[12] The HI person would then go on, rather annoyingly, to prove that 47% of the users will operate the device in precisely that way – turning it on and then adjusting the pot – and that 13% of those customers will return the unit without trying to figure it out, costing your company millions of dollars… and you your bonus. The lessons are: (a) Never *ever* ship a consumer product without a human interface tester's blessing. (And that way, they take the blame.) (b) Analog designs are not as easy to simulate in the digital domain as they might first appear.

F. Design review

After the embarrassing HI experience, a full design review was scheduled. Someone remarked that anything sitting on the sun-baked sand would get very hot, to which someone else replied "Tell the user, who's sitting on a beach chair, to put the monitor in the shade underneath." This suggestion received a (wordless) withering stare. Moving along, the design temperature was set at 85°C. That's pretty hot, and (with a doubling every 10°C) the BSS123's leakage current would rise to an unacceptable 3.2 μA (64× greater than the datasheet's 50 nA typ at 25°C). Oops! Interestingly, the μC's comparator is OK, because its 50 nA input spec extends to 85°C; similarly the photosensor's dark current (50 pA at room temperature) would increase to about 3 nA, also OK.

Exercise 15.1. Find an alternative to the BSS123 MOSFET reset switch: one possibility is to adapt the method of Figure 13.49 to a microcontroller. *Hint*: Replace U_2, U_3, and U_4 with the processor. Recall that the voltage at "X" doesn't matter. In that design we suggested an LMC6842 op-amp, which specifies 10 pA maximum input bias at 85°C; that's fine here, but the op-amp's supply current is 1 mA, dominating the battery drain. See if you can find a better choice.

15.3 Overview of popular microcontroller families

For the simple suntan monitor, we chose a controller from the popular Atmel 8-bit AVR series. We like them, because they exemplify a clean architecture, with plenty of general-purpose registers and a flat address space. They come in multiple package styles, including the through-hole dual in-line package (DIP) that we favor for quick prototyping. They are also well supported in terms of software tools, including open-source C-language compilers that are not too encumbered with controller-specific idiosyncrasies.

What other controllers are out there? We offer a listing of contemporary offerings below, with a bit of

[11] For the AVR these include brownout voltage level, on-chip debug on/off, clock source (internal, external square wave, external crystal, etc.), JTAG enable, SPI enable, and watchdog timer enable.

[12] The software could, of course, be modified to "pay attention."

commentary. This should be considered a snapshot in time, as microcontrollers constitute one of the faster-changing areas of electronic technology.[13] You might keep in mind the characteristics of an ideal microcontroller:

- plenty of on-chip flash (reprogrammable program memory) and RAM
- small versions available in easily prototyped DIP (or SMT on DIP adapter)
- internal oscillator
- minimum external parts count to get started
- fast, low power consumption, wide supply voltage range
- inexpensive programmer, serial or USB
- in-circuit programming
- free programming software
- free assemblers and compilers
- free Integrated Development Environment (IDE) to tie it all together
- in-circuit debug/emulate through the IDE
- open-source toolchain that runs in Windows, Mac, Linux
- active user community

AVR *(Atmel)* 8-bit RISC[14]; clean architecture, 32 general-purpose registers; in-circuit debugging; open-source (Linux, GCC, Arduino) tools supported by Atmel and integrated into their software; some chips include USB; many package styles (including DIP); available as hybrid/FPGA (Atmel); least expensive, less than $0.50; competes with Microchip's PIC. 32-bit series: AVR32.
Programmers: simple serial ($12); Atmel AVR Dragon USB in-circuit debugger ($50). Arduino boards with USB power and programming are $20–$30. AVR Butterfly board with LCD and buzzer ($20). *Our recommendation for getting started.*

PIC *(Microchip)* 8, 16, and 32-bit; high-speed and low-power versions; some in-circuit debugging; many development kits; many package styles (including DIP); FPGA soft core; long time favorite of hobbyists; least expensive, less than $0.50.
8-bit: PIC10F, 12F, 16F, 18F. 16-bit: PIC24, dsPIC30, dsPIC33. 32-bit: PIC32MX.
Some chips include integrated USB or Ethernet MAC and PHY. Free chip samples from Microchip. The older 8-bit families, especially PIC16F84, used to be extremely popular (newer compatible versions have internal oscillators). Assembly-language coding can be tricky, owing to memory banking, a single accumulator, conditional skip (rather than jump) instructions, and an 8-level call stack in hardware (not RAM); but timing of instructions and interrupts is very predictable.
The 10F–16F devices are partially supported by C compilers by SourceBoost and Hi-Tech C, with prices ranging from free (limited versions) to $3000. Only 18F and above are well suited for C programming, and are supported by Microchip's C compilers (a free limited version is available to students). The 16-bit devices have the stack in RAM and no bank switching, and can be programmed with a variant of the GCC C-compiler. The 32-bit devices use an industry standard MIPS core (similar in features to the ARM), with a Microchip C compiler based on GCC.

ARM *(multivendor)* 32-bit RISC with optional 16-bit instructions; ARM Cortex M3, ARM7, and ARM9 series; least expensive, around $2; high-performance; in-circuit debugging; large range of included functions and I/O; open-source C/C++ with GCC and Eclipse debugging GUI (graphical user interface), also Arduino software for ARM-based *Due*; flat memory model that is easy to program; pincount $\geq$28 only (no DIP). Some can be programmed through USB, but need JTAG pod ($50) to debug. Available as prebuilt header boards from multiple vendors ($30), protoboards ($60); also Arduino Due (pronounced "doo-way," $50), UDOO (udoo.org, $99), Raspberry Pi ($30), Odroid ($65), and others.
Manufacturers: ADI, Atmel, Broadcom, Cypress, Freescale, Infineon, Microsemi, NXP, Renesas, Samsung, Silicon Labs, ST, TI, Toshiba; available as hybrid/FPGA (Altera SoC series, Xilinx Zynq series). From 8-bit replacements to iPhones. Pipelining and complex instructions provide lots of speed, but less predictable running time. *Would be our recommendation for getting started, if prototyping a design on your own board were easier.*

8051 *(multivendor)* 8-bit; evolution of early μC; mature development tools and development kits; some quirkiness in C-code, owing to complex address space; some in-circuit debugging (Silicon Labs); legacy instruction set limits memory, register, and I/O options; many package styles (including DIP); FPGA soft core. Good free development software (limited), e.g., "Ride" IDE (www.raisonance.com): C-compiler, assembler, simulator.

Rabbit *(Rabbit)* Emphasis on Ethernet and WiFi; available as modules, or as bare chips; C-language IDE; development kits and tools; popular.

[13] Stay abreast by looking at magazines like *Circuit Cellar*, *EDN*, and *Electronic Products*; distributors like DigiKey and Mouser; or do a search for "<μCname> tutorial" or "getting started with <μCname>."

[14] Reduced instruction-set computer; the opposite is a complex instruction set, CISC.

MSP430 *(TI)* Micropower 16-bit RISC; popular for low-power, RF, and LCD controllers; many package styles (including DIP). JTAG programming ($75–$100); open-source compiler MSPGCC and free code-size-limited versions of commercial compilers.

SH-4 *(Renesas, formerly Hitachi)* 32-bit RISC; smallest package is QFP-208 (LQFP-48 for the SH-2); popular in motor and engine control. M16C/R8C are 16-bit, available in DIP.

Coldfire *(Freescale, formerly Motorola)* Embedded 32-bit 68000-like architecture; no packages smaller than 64-LQFP; GNU C compiler; development kits (Tower System).

ST6/7 *(STMicroelectronics)* 8-bit, available in DIP; ST9/10 are 16-bit, smallest package LQFP-64. STR7/9/32 are 32-bit ARM7/9/Cortex, smallest package VFQFN-36 or LQFP-48.

PowerPC and MIPS *(multivendor)* Used in high-end embedded systems such as automobiles, networking, and video; by contrast, ARM dominates in cellphones and PDAs.

Blackfin *(Analog Devices)* 16/32-bit RISC plus DSP; fast and high-end, optimized for audio, video, and imaging. Easier than some other high-end controllers. Open-source GNU compilers, uClinux, FreeRTOS.

Propeller *(Parallax)* 8-core, 32-bit parallel processors; dc–80 MHz; small line of DIP-40/44 or SMD; philosophy of "why have lots of built-in hardware peripherals like UARTs and SPI when you can put eight real processors in there, give them all access to the I/O, and write good libraries to bitbang everything from mice and keyboard interface to analog video (VGA/NTSC)."

PSoC *(Cypress)* 8-bit microcontroller with configurable analog blocks: Op-amp, ADC/DAC, filter, modulator, correlator, peak detector, etc.

XMOS *(XMOS Ltd.)* Event-driven multithreaded 32-bit processors, with events and threading in hardware up to 400 MIPS; programming in XC, C/C++, assembly; free development tools.

15.3.1 On-chip peripherals

All microcontroller families include on-chip program and data storage, and timer/counters. They generally offer a selection of options such as comparator, ADC, and DAC; I^2C, SPI, and CAN buses; UARTs, USB and Ethernet; and support for external LCD, pulse-width modulation (PWM), video, and wireless devices. Here is a listing of some on-chip or low-level support found in contemporary microcontrollers, roughly in order of increasing level of complexity:

A. Low complexity

ADC, DAC, Analog Comparator
CANbus
Debug (JTAG or proprietary 1- or 2-wire interface)
I^2C / SMBus / TWI ("Two-Wire Interface")
Interrupts, sometimes with priority
Keypad matrix scan
Keyboard (serial), mouse
LCD (bare)
PWM
Real-Time Clock
SIM Card / Smart Card serial interface
Synchronous Serial Ports (SSPs): SSI, SPI, SSI Microwire
Timer, counters, watchdog
UART (RS232, RS485, IrDA), some with modem control

B. Mid-level complexity

AC97 (Intel Audio 20-bit 96 ksps stereo PCM)
Bluetooth wireless
Compact Flash (CF card)
Ethernet
External SRAM
GPS
I^2S (Inter-IC Sound: CD to DAC at 2.8 Mbps)
IrDA (high speed, to 4 Mbps)
Motor control, shaft encoder
PCMCIA[15] ("PC card")
S/PDIF (AES/EBU) audio (Dolby Digital or DTS surround)
SD/MMC flash cards
USB 1.1 (Host or Device, to 12 Mbps)
ZigBee wireless

C. High complexity

Camera and image sensors (CMOS, CCD)
External DRAM/SDRAM
Graphics display (bare color LCD)
MMU (Memory Management Unit) with OS protec
MPEG4 encode/decode
Operating System (Linux, Windows CE, Palm)
PCI, PCIe

[15] Sometimes translated as "Personal Computer Manufacturers Can't Invent Acronyms."

Storage Drives: ATA, IDE, SATA
USB 2.0 high speed (480 Mbps)
Video (NTSC, PAL, VGA, DVI, DV)
WiFi (802.11)

We predict, with considerable confidence, that the list will expand and that some items will be demoted.

We turn now to four more design examples to illustrate some common ways in which microcontrollers are used in embedded circuit design. These examples deal with
(a) ac power control (in which a microcontroller switches line power, under the direction of a serial link);
(b) a frequency synthesizer (in which a microcontroller runs errands between a user and a direct-synthesis chip that only understands cryptic serial commands);
(c) a precision temperature controller;
(d) a stabilized 2-wheel pseudo-Segway™contraption (which its inventor calls a "Psegué").

15.4 Design example 2: ac power control

At our astronomical observatory we have a dozen or more ac-powered devices that we need to control remotely – things like dome and telescope drives, mirror heaters, lights and cameras, power supplies for detectors and processors, and the like. Building a circuit to do this is a breeze with a microcontroller. And a benefit to having some intelligence in the control box is that it can remember a default configuration, and it can report status information (which outputs are powered, how much current they are using, and so on).

Let's do a fairly simple example here, namely a controller to switch 110 Vac power on two outlets, with readback of the commanded state, and with confirming readback that the actual ac output is present. In addition to remote control via serial RS-232 or USB, it will also permit "local" control via switches on its panel; some LEDs there will indicate which outputs are powered.[16]

15.4.1 Microcontroller implementation

Figure 15.4 shows the circuit. For variety we chose a modest member of the popular PIC family; the particular part (PIC16F627, DIP-18, $2) includes the needed serial port UART (universal asynchronous receiver–transmitter), an internal oscillator, nonvolatile data memory (EEPROM) that guarantees endurance of a million erase/write cycles, and plenty of digital port pins. We don't need much of anything else that the manufacturer has thrown in (analog multiplexer, comparator, PWM hardware, 20 MHz speed), but the microcontroller costs just $1.50, a small fraction of the total parts cost, so, hey, why complain.

The easiest way to switch ac power from a logic signal is to use a *solid-state relay* (SSR; see §§12.7.5 and 12.7.6), which consists of a sealed module containing an optocoupled SCR or triac. These devices are available from many manufacturers.[17] They accept a logic-level dc drive (often specified as 3–15 Vdc, or 3–32 Vdc), drawing somewhere in the range of 3–15 mA. The output is well isolated (typically specified as 3.5–5 kV), and most SSRs implement zero-voltage turn-on and zero-current turn-off. Typical load ratings are 10 A or 20 A at 280 Vac, but you can get standard units up to 100 A and 660 Vac.

In our example we've use a digital port bit to drive the SSR, configured to power on logic LOW; that choice of polarity is good here, because the port pins specify a typical saturation voltage of ∼0.35 V when sinking the required 15 mA, versus ∼1.3 V (below the positive rail) when *sourcing* that current. A panel LED rides along, indicating relay drive.

Solid-state relays have some leakage current when off, typically specified in the range of 0.1–10 mA; so we've bridged the switched ac with a 15k resistor.[18] The ∼8 mA current through it, when on, is used to light a panel LED (indicating delivery of ac to the load), and also to drive an opto-isolator to signal the microcontroller that all is well. This optocoupler is designed for ac operation, with its internal back-to-back LEDs; this particular unit has 5 kV isolation, 50% minimum current transfer ratio (ratio of phototransistor output current to LED drive current), and costs about $0.25 in moderate quantity.

Exercise 15.2. What is the purpose of the 1k resistor? Why is a diode needed across the indicator LED?

Moving back to the microcontroller circuitry, we've taken advantage of the internal "weak pullup" digital input mode (specified as 50–400 μA) for the input switches: a momentary contact pushbutton (for initiating local control), and a pair of spring-return momentary-contact lever

[16] If you want to save time (and cheat yourself out of an education), you can instead just throw money at the problem: there are nicely engineered units to do this job, for example from Pulizzi Engineering (Eaton Corporation) or APC (Schneider Electric). We opted for the educational opportunity, thanks in part to a grant from the Shulsky Foundation.

[17] For example, Crouzet, Crydom, Magnecraft, Omron, Opto22, P&B, and Teledyne.

[18] We measured the off-state leakage for two SSRs, and found ∼2 mA and ∼6 μA for two Crydom 240 Vac SSRs (a D2425 and EZ240D18, rated at 25 A and 18 A, respectively). They specify worst-case leakages of 10 mA and 100 μA, respectively.

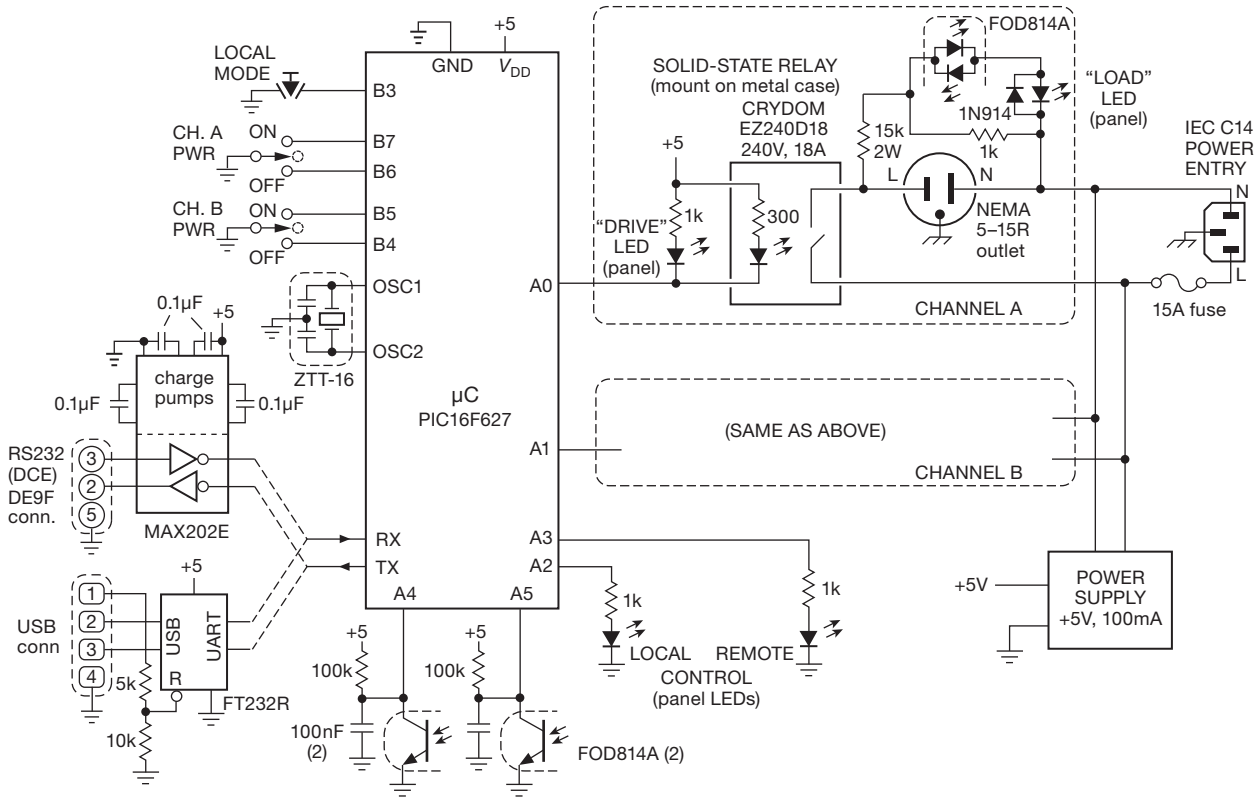


Figure 15.4. Dual ac-power control box, with remote (serial) or local controls. The SSR dissipates 10 W at a load current of 10 A, so it needs adequate heat-sinking. A mechanical relay could be used instead (e.g., Panasonic ALE12B05, contact rating of 277 Vac and 16 A, with faston tabs like the Crydom SSR). For the latter you'll need a transistor to drive its 80 mA coil; but the good news is its price: $1.60 in unit quantities, compared with $35 for the SSR.

switches for commanding the outputs on or off while in local control. A pair of panel LEDs indicate whether the unit is under local control (i.e., panel switches) or remote control (i.e., from a host computer, connected to this device via RS-232 or USB serial links).

We chose this particular controller partly on the basis of its internal oscillator. . . and then we discovered that it isn't accurate enough! The specified accuracy is ±7%, not good enough for 8N1 asynchronous serial communication: as explained in §14.7.8, the receiving device has to sample the 8 data bits in the middle of each bit cell, using its own asynchronous baud clock, having synchronized to the START bit at the beginning of each byte. So a 7% clock rate error between transmitter and receiver slides the last (8th) bit sample clear into the preceding or following bit cell.

The solution is either to use an external oscillator of adequate accuracy (for example, a crystal oscillator module, which costs about $2), or to put an external bare crystal or ceramic resonator across the OSC1 and OSC2 pins that the microcontroller provides for this purpose. We chose the latter, using a convenient 3-pin resonator that includes the necessary capacitors; this puppy costs about $0.40, with a specified accuracy of ±0.5%.

This microcontroller has an internal UART that makes serial communication pretty simple.[19] The only external hardware needed to connect directly to a host computer RS-232 "COM" port is a level translator (RS-232 levels are bipolarity; see §14.7.8). We chose the popular MAX202, which runs from the single +5 V supply, and creates the needed ±10 V with an internal charge pump voltage doubler–inverter; it requires four 0.1 µF capacitors.

[19] Although getting any microcontroller's UART settings right (timer mode and division ratios, serial data mode, interrupts, enables) is usually tricky. The best bet is to copy, shamelessly, the settings from the example that is invariably given in the applications section of the datasheet (more like a data*book*: typically 300–400 pages!), provided, no doubt, in the expectation that you stand only a small chance otherwise of getting it right.

If you don't want to be bothered with such details, you can spring for the MAX203 with internal capacitors; but you'll pay about $5, compared with $0.75 for the bare-bones translator. Faced with this choice, most designers go the cheaper route.

If you want to communicate via USB, you've got several choices. You could choose a microcontroller with internal USB support, for example the PIC18F2450. The complexities of USB should not be underestimated, however. You need to deal with USB protocols, either with custom software drivers, or implemented using one of the sixteen established "device classes" such as human interface device (keyboard, mouse), mass-storage device (flash or disk memory), printer, imaging device (camera), communications device (Ethernet). An attractive and simpler approach is to stick with the asynchronous serial design and use a well-engineered interface chip such as the FT232R, which is intended for exactly this kind of application and which costs about $4.[20] It has the virtue of working well, and it comes with all the necessary drivers and other software for the host computer; that's what you use initially to set up parameters like baud rate and mode, which the chip's EEPROM then remembers.

Likewise you can adapt this circuit's native UART interface to Ethernet, with the use of a commercial Ethernet-to-serial module like the Lantronix XPort. This elegant device looks like a somewhat elongated RJ45 (Ethernet jack), with all the electronics packed inside; it even has the usual green and yellow network activity indicator lights. It costs about $50.

We end the circuit tour with the microcontroller end of the opto-isolators. These have a floating phototransistor output, which we use to pull a pair of digital port input bits LOW when LED current flows at the ac-sensing end. (The LEDs shut off for a portion of each half-cycle, of order 1 ms, hence the 100 nF capacitors to hold the digital inputs LOW during those brief interruptions.) It's tempting to exploit the internal "weak" pullups, but they're not weak enough: according to the specifications, the pullup current can be as large as 0.4 mA, which would require 1 μF capacitors to keep the voltage at the input ports below 0.4 V during the $\sim$1 ms gaps. The solution here is to repudiate such bullying tactics: to regain control of the situation we turned off the internal pullups, and used "even weaker" external pullups whose 50 μA current requires only 0.1 μF capacitors.

15.4.2 Microcontroller code

The coding for the ac power control is straightforward. Take a look at the Pseudocode 15.2: at power-up you disassert port bits that would energize the relays, configure the port bits (as inputs or outputs, and with or without pullups), set up the serial-port baud rate and mode, and enable the hardware reset both on power dip ("brownout") and on program crash ("watchdog"). Then the monotonous daily life of this microcontroller consists of perpetual polling of the (not-debounced) switch inputs, looking for any change of state from the previous scan, along with a kick to the watchdog.[21] (It's a good thing that microcontrollers don't get bored.) Assertion of the LOCAL-mode pushbutton causes the device to go into local mode, in which other buttons are acted on, as indicated. The program in Pseudocode 15.2 performs a "software debounce" by delaying the next scan of switch inputs for 10 ms following the first detection of any change.

This main loop runs forever. But the serial port has not been ignored: during setup, the UART has been configured to generate an interrupt on receipt of an input character. So an incoming command on the serial port causes program diversion to the interrupt handler, where the seven possible commands (each consisting of a single byte printable character) are parsed and acted on. Those actions mostly consist of setting or clearing a single bit (either in a port, to control a relay; or a register, to set the local/remote mode). The "s" (request for status) command is more interesting, requiring the processor to collect the four bits indicating relay status (both the commanded output bit that drives the relay, and the optocoupled ac indication, for each of the two channels), and the mode bit; those are masked and shifted into a register, then sent back as a single binary byte.

A. Some comments

(a) Microcontrollers generally include some non-volatile data memory, which could be used to save the state of the outputs in the event of power failure or reboot; this controller has 128 bytes of EEPROM for this purpose.

[20] You can get it in several physical incarnations, for example, a cable with a USB connector on one end (in which the chip is concealed) and a 9-pin serial connector (with RS-232 voltage levels) on the other; alternatively, you can get it with flying leads at the serial end, configured for 3.3 V or 5 V logic-level operation, upgraded part. The upgraded part number is FT2232.

[21] Most engineers persist in calling this "kicking the dog." But our quirky colleague Frank Cunningham has pointed out that what you're really doing is *petting* the dog: when you kick a dog, it barks immediately; when you pet a dog, it barks only when you stop petting it.

Pseudocode 15.2 ac power control pseudocode

Setup

`Ports:` disassert (setbits) relay outputs (A0, A1)
setports LEDs (A2, A3) and relays (A0, A1) as outputs
setports switches (B3–B7) and opto (A4, A5) as inputs
set switch input ports (B3–B7) as weak pullups
set up UART: baudrate, 8N1, interrupts, enable
`Low Power:` disable unused peripherals
`Automatic Reboot:` enable "brownout reset" and watchdog

Switch Polling Loop

read switches (port B)
if (any switchbit has changed)
if localmode switch (B3) asserted, set mode=local
if mode=local and any switch went H→L
turn ON or OFF that relay output
delay 10 ms ("software debounce")
kick the watchdog
repeat **Switch Polling Loop**

Serial Interrupt Handler

get character
if "r" or "R", set mode=remote
if "l" or "L", set mode=local
if "s" or "S", assemble 5-bit status byte and send
if mode=remote
if "A", turn on A relay
if "a", turn off A relay
if "B", turn on B relay
if "b", turn off B relay

(b) It is best to keep interrupt service routines short and simple; in particular, you would avoid tasks that tie up the processor in the service routine while waiting for a slow peripheral to complete (e.g., a serial port!), because that may cause the main loop to miss a switch-input event. Here it's OK, though, because every serial-port command and every response is a single byte, so the routine does not have to wait for a busy flag to clear.[22]

[22] There is a standard, known as SCPI (Standard Commands for Programmable Instruments), that specifies a set of commands, and a syntax, for communicating with programmable instruments. SCPI is a bit wordy, but it is clear, and it has been widely adopted by manufacturers of electronic instruments (e.g., Agilent, Fluke, Keithley, LeCroy, Rohde & Schwarz, and Tektronix). For example, to read a voltage you could send the command `MEASure:VOLTage:DC?` (the lowercase characters are optional). The measurement response is returned in a defined format.

15.5 Design example 3: frequency synthesizer

Our next example illustrates a common application of embedded microcontrollers, namely their use to manage communications between an instrument panel (or computer interface) and some circuitry that requires digital programming and control. Figure 15.5 shows a two-channel frequency synthesizer, using one of the elegant direct digital synthesis (DDS) chips from Analog Devices (§7.1.8). We've chosen the AD9954 (one of the three dozen current DDS offerings), which synthesizes a high-quality sinewave via internal table lookup and D/A conversion. It can generate output frequencies from dc to 160 MHz, with resolution set by the 32-bit "tuning word"; that corresponds to ∼0.1 Hz when a 400 MHz internal oscillator is used to clock the chip. Although you can get faster DDS chips, this one has excellent spectral purity, owing to its 14-bit output DAC; this translates to a clean output sinewave,

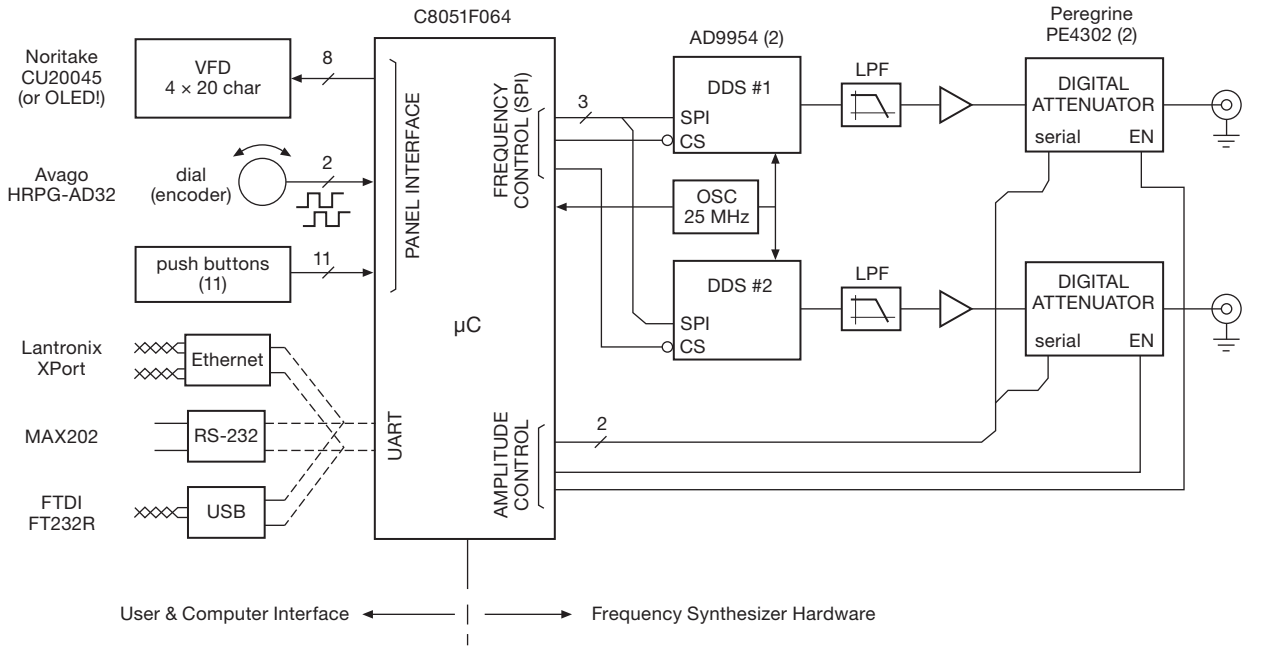


Figure 15.5. A microcontroller is ideal for interfacing between digitally programmed chips (here a pair of DDS frequency synthesizers, with programmable attenuators) and a set of human-operated panel controls and displays. And the μC's serial interface makes it easy to add remote computer control (here a pair of DDS frequency synthesizers, with programmable attenuators, see Figure 15.6).

with low phase noise and "spurs" (spurious frequency components).

This DDS chip, which costs about $25, won't do much of anything until you program it through its 4-wire SPI port. You could imagine interfacing the DDS to a laptop computer and running some software on it so you can program the chip for the frequency you want. But what you'd really like is a nice set of human-usable panel controls – a knob to dial up the frequency and amplitude and a numeric display to show those values (Figure 15.6). That's best done with an embedded microcontroller. And, of course, that controller can accept inputs via a USB or Ethernet port, so once you've built this instrument you can add remote digital control as well.

The block diagram (Figure 15.5) shows the microcontroller riding herd over the DDS chips (via their SPI serial ports) and the digital attenuators (via a similar 2-wire protocol, described in their datasheet). The synthesizer chips need an accurate 400 MHz clock reference. This is generated by its internal phase-locked loop (PLL) frequency multiplier from an external lower-frequency reference; we chose a 25 MHz crystal oscillator (with $\times 16$ on-chip multiplication), which conveniently can clock the microcontroller as well.

On the user side there are buttons and a dial (an optical "incremental rotary encoder") for entering and changing parameters (frequency, amplitude, phase); and there's a vacuum fluorescent 4-line character display (VFD) that is pin compatible with the more usual LCD shown in the photograph, for telling the user what's going on. VFDs cost more than LCDs, but they are gorgeous. Remote control communicates through the on-chip serial UART, which requires only a voltage translation for a legacy RS-232 port; or you can use a smart serial format conversion device like the FT-232 from FTDI (for USB) or the XPort from Lantronix (for Ethernet).[23]

Although shown casually in the block diagram, there is additional detail in these user interface connections. The rotary encoder, for example, generates a pair of square waves, in 90° quadrature, from which you can figure out in which direction (and by how much) it has rotated. Inexpensive encoders use mechanical contacts, so you have to worry about the usual contact bounce. We've gone first class here, though, with an optical encoder (it lasts forever) that generates clean logic-level outputs (no bounce!). The

[23] Additional Ethernet interface devices are available from companies such as Digi International, Ipsil, Connect One, and WIZnet.

corresponding firmware simply notes the level on the other line when it senses a transition on either of the two inputs.

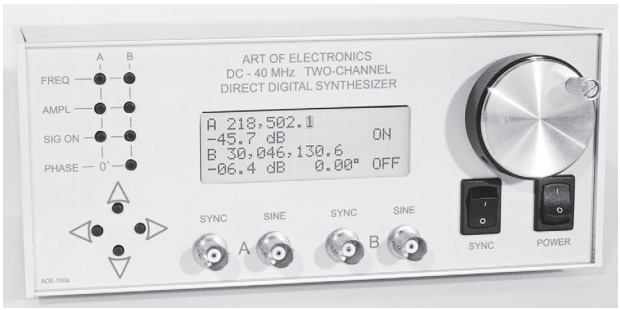


Figure 15.6. Dual DDS frequency synthesizer of Figure 15.5, with panel controls (eleven buttons and one rotary encoder dial) and readout. (Designed, built, coded, and documented with excellence by Jonathan Wolff; see www.artofelectronics.com/synthesizer)

The pushbuttons are mechanical, and prone to contact bounce; they must be debounced in software, as with the previous ac power control example. We've connected each button to its own input port pin (configured with weak internal pullup), which is the simplest configuration (though not the most economical). This particular microcontroller has 59 digital port pins scattered around its 100-pin package (Figure 15.7), so we're unlikely to run out. The alternative is a matrix-switch configuration, with row and column lines that are bridged by the individual switches, which here would require four fewer pins; we'll discuss that, along with other wrinkles, shortly.

15.5.1 Microcontroller code

The microcontroller's tasks are, briefly:
(a) set-up port bit modes, UART parameters and interrupt, SPI port, and watchdog;
(b) initialize DDS chips and attenuators to last stored state (held in nonvolatile memory), and send current operating parameters (frequency, amplitude, phase) to VFD display;
(c) execute forever a main loop that kicks the watchdog, checks for rotary dial movement, for button actuations, and for a serial command flagbit (set by the serial interrupt handler), and acts accordingly;
(d) service UART interrupts (independent of the main loop) by appending characters to a line buffer, and setting a software flag ("serial_cmd") upon receipt of a newline character (end of command).

The listing in Pseudocode 15.3, though somewhat abbreviated for this example, spells this out in greater detail.

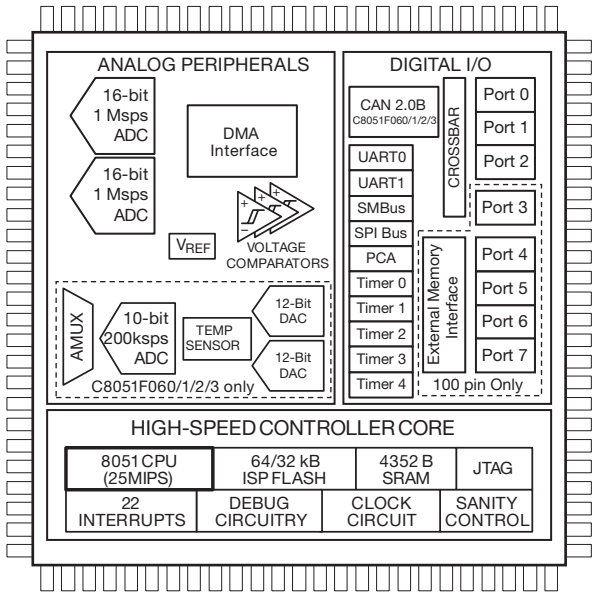


Figure 15.7. The venerable 8051 has come a long way since its 1980 debut, with many added functions, and one clock cycle per instruction (the original required 12) in contemporary variants like the C8051F060 series from Silicon Labs, from which this diagram is adapted, with their permission. (Curiously, the 328-page datasheet reveals nothing about the meaning or function of "sanity control.")

A. Some comments (Hardware)

(a) The block diagram shows relatively simple amplitude control, with a digital attenuator providing 0–30 dB attenuation in 0.5 dB steps. You can do much better, in range, step size, and calibration, by using the intelligence and memory of the microcontroller: the synthesizer in the photograph uses two additional layers of amplitude control, namely a fixed 30 dB attenuator that is switched in to reach 60 dB, and fine adjustment ($\pm 6\%$) of the DDS chip's "reference current" (via an EEpot) to provide 0.1 dB resolution. One further trick is needed to make this all worthwhile: the user initially calibrates the instrument, by reporting measured signal levels at several frequency and amplitude settings over the full range, as prompted by the firmware; the microcontroller stores this calibration data in nonvolatile memory, retrieving it to produce a linear and accurate output-amplitude characteristic.

(b) The DDS output waveform updates at the internal reference clock frequency, in this case f_{ref}=400 MHz, generating successive samples of the output "staircase" approximation to a sinewave. The resultant output signal, settable from 0 to 160 MHz, contains spurious out-of-band frequency components, the lowest of which is at

Pseudocode 15.3 Synthesizer pseudocode

```
Setup
    Ports: setports pushbuttons and rotary dial as input
        setports to attenuator as outputs
        setports to VFD as outputs or inputs
        set up UART: baudrate, 8N1, interrupts, enable
        set up SPI port
    Automatic Reboot: enable brownout and watchdog timer (timeout=1s)
    Read Stored State: copy NV-stored state to active registers
        clear and reload VFD and DDS from active registers

Main Loop
    kick the watchdog
    read switches and rotary dial
    if (any debounced switchbit has changed)
        if (freq, ampl, or phase), set new mode
        if (up/down arrow or rotary dial), increment/decrement that register
            refresh DDS or attenuator, and display
    if (serial_cmd bit) parse command, clear serial_cmd bit
            increment/decrement corresponding register
            refresh DDS or attenuator, and display
    repeat Main Loop

Serial Interrupt Handler
    append character to line buffer
    if newline, set flagbit serial_cmd
```

$f_{spur}=f_{ref}-f_{out}$. That's why lowpass filters are needed at the DDS's DAC outputs; typically you'd use an *LC* multisection elliptic filter with a sharp cutoff characteristic at $f_c \sim 180$ MHz. The DAC outputs are differential *currents*, best filtered with a symmetrical differential filter, an example of which is shown in the 40-page datasheet.

(c) With dual DDS chips set to the same frequency, you can control the phase difference between the two outputs, providing the chips have a common reference clock. This particular DDS accepts a 14-bit phase offset word, equivalent to steps of 0.22° in relative phase.

(d) We used individual port input bits for each pushbutton, with a common ground return, a perfectly reasonable approach here with only 11 buttons, and a microcontroller with 100 pins. The alternative is to use a row and column *matrix*, with the switches bridging the intersections (Figure 15.8). Here the column lines are driven by output pins, and the rows are connected as inputs with internal pullups. To poll the switches the microcontroller pulls the column lines successively LOW, observing the returned input levels from the row lines; debouncing is done in software, as it was with dedicated input lines. Matrix encoding is efficient when you have many switches, because the number of I/O pins used is the sum of the number of rows and columns, rather than their product. In some cases, moreover, you are forced to do it this way, because the switch array is built as a matrix; an example is a 4×4 hexadecimal keypad. One caution: a matrixed keypad generates false outputs if three (or more) keys are pressed simultaneously; figure this out for yourself.

B. Some comments (Firmware)

(a) Having a microcontroller in charge provides a nice opportunity to add features and functions that you wouldn't even think of otherwise. For example, the synthesizer in the photograph implements a nice algorithm (which we have not seen elsewhere) for determining how the digits are controlled by the dial and the buttons. The left- and right-arrow buttons do the obvious thing, namely choosing the ("active") digit being modified; and the up and down buttons increment and decrement that digit (with carry or

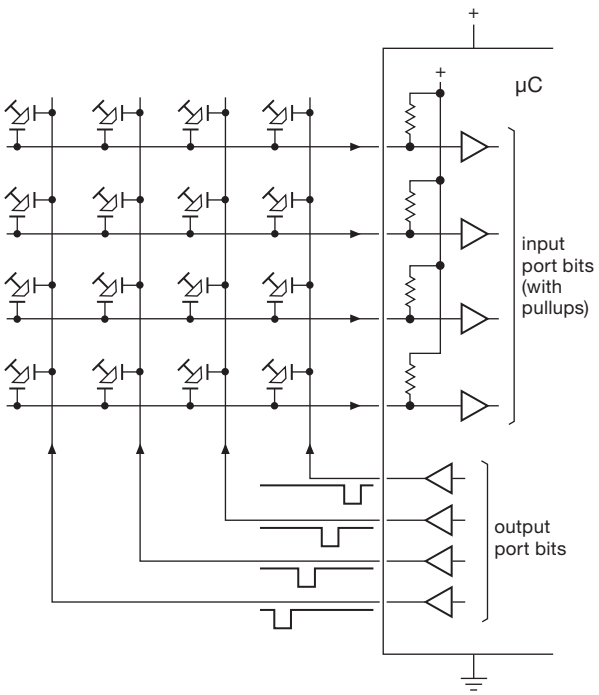


Figure 15.8. An array of switches can be polled in a row and column matrix arrangement, here illustrated with a 4×4 keypad. This reduces the number of port pins to $N_{\text{matrix}} = r + c$, compared with the $N_{\text{indiv}} = r \times c$ required for individual dedicated readouts.

borrow), assuming that you want to set an exact frequency. But the dial is treated differently: if you move the active digit leftward, the dial acts on that digit and zeros all digits to the right, on the assumption that you want to move the frequency faster, and without the baggage of leftover less-significant digits. On the other hand, moving the active digit rightward and spinning the dial preserves the more significant digits to the left, on the assumption that you are zeroing in on the target frequency. We wish commercial instruments would use this sensible algorithm!

(b) Just as with a computer's mouse, the panel dial's motion is translated by an acceleration algorithm, so that a fast spin of the dial takes you farther and faster.

(c) The firmware takes care of capturing initial calibration data and interpolating from that data during subsequent operation, as described above. A microcontroller lets you program frequency sweeps – linear ramps, cyclic ramps, or nonlinear sweeps (e.g., a logarithmic sweep, with equal time per octave).

(d) And, of course, a reprogrammable microcontroller lets you fix bugs, and dream up new features (with new bugs).

15.6 Design example 4: thermal controller

Here's an example of embedded control in the service of, well, *control*: imagine we want to hold steady the temperature of a well-stirred liquid bath, in which we've put both a heating element and an accurate temperature sensor. At first glance the solution seems easy: just apply plenty of feedback, using as the error signal the difference between the measured temperature (as reported by the sensor) and the desired target temperature. Raise the loop gain until the temperature error is small enough, just as you would with an op-amp voltage amplifier.

This is a classic "control" problem, encountered in industrial settings such as chemical plants (temperature and flow control), manufacturing and robotics (motion control), and the like (Figure 15.9). Simple feedback works poorly in such situations, because the controlled system has time delays (and therefore lagging phase shifts) that promote oscillation; and reducing the loop gain to prevent such oscillation leaves too little loop gain, so the controlled system both deviates from the desired condition and moves slowly to correct disturbances.

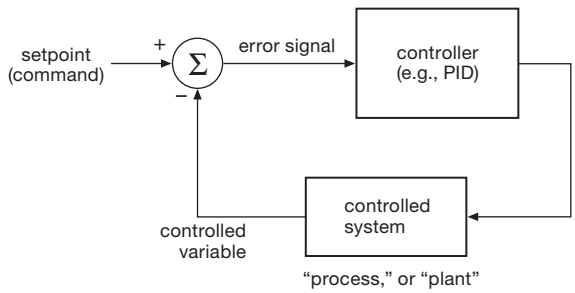


Figure 15.9. Classic control (or "servo-") system. For our example the setpoint is the desired temperature, the "plant" is the heater–waterbath–sensor, and the controller is implemented with a microcontroller. The μC's input compares the desired and actual temperature, and its PWM output powers the immersion heater.

There is a large literature on this subject, and on the feedback principles underlying the "tuning" of a controller to match the properties of the controlled system (called, curiously, the "plant," probably in a throwback to the industrial revolution). The general approach is to construct (with analog or digital methods) a so-called "PID" (proportional, integral, differential) controller, in which the error signal versus time is subjected to those three processes, and then combined in carefully chosen proportions to create the correction signal that is applied to the controlled system. The procedure for tuning the coefficients is basically straightforward (though it is considered by many to be a mysterious art), and is best understood with a Bode plot like those in Chapter 4. Although it's easy to make amplifiers,

integrators, and differentiators with op-amps, microcontrollers are naturally suited to PID control; they have added benefits, too, such as the ability to report status, to accept and modify tuning parameters, and to carry out nonlinear control algorithms.

15.6.1 The hardware

Popular temperature sensors include a thermocouple (a junction of two dissimilar metals, which generates a voltage, typically 20–40μV/°C), a thermistor (a resistive material of negative temperature coefficient, typically −4%/°C), a silicon-based temperature sensor IC (that exploits the −2.1 mV/°C negative tempco of a diode's forward drop), or a platinum RTD ("Resistance Temperature Detector," a wirewound resistor, usually standardized at 100 Ω at 0°C, whose resistance increases by 0.385%/°C). These sensors differ in temperature range, accuracy and stability, speed of response, size, and cost. For this example (and without excessively opening this substantial can of worms) we chose the RTD, primarily for its good stability; it also has excellent linearity, and a wide operating temperature range (−200°C to +600°C). As with other temperature sensors, it is available in a nice waterproof probe assembly. At the other end – the heater – a power resistor is all you need, once again in a submersible package (an "immersion heater").[24]

There are several possibilities for the heater drive. The simplest way is to turn it on when you need heat, and off when you don't. This is called a "bang-bang" controller, and it's the way home heating systems work. It's guaranteed to produce a cycling of the temperature about the target, which is less than ideal. It is efficient, however, because the drive transistor operates as a switch, with very little power dissipation.

A better way, if the system permits, is proportional control. Here we could use a linear power output stage, varying the heater drive in a continuous manner, according to the controller's command. The drawback is that the linear driver stage has to dissipate lots of power. This is where pulse width modulation fits in. By operating the output driver as a switch, at, say, 10 kHz,[25] and varying the duty cycle (the ON fraction of each switch cycle), we get the best of both worlds: proportional control, and minimal driver dissipation. For this reason PWM is popular in other "linear" applications, for example audio amplifiers (see §2.4.1C) or motor drives.

Figure 15.10 shows the circuitry for a microcontroller temperature controller. Looks simple enough, but as usual the devil (and the joy) is in the details.

A. The sensor

We chose a generous −50°C to +150°C nominal range of control, over which the standardized "100 Ω" RTD temperature probe's resistance goes from 80 Ω to 160 Ω, approximately.[26] To keep the effects of self-heating negligible, we used a 2 mA bias current, which produces corresponding voltages of 160 mV and 320 mV across the RTD, and a maximum self-heating dissipation of just 0.6 mW.

These are smallish voltages, compared with typical ADC conversion ranges of 1.25 V, 2.5 V or 5 V; so we need some gain, and we need an offset, to match the ADC's full-scale span. A bridge arrangement, as shown, produces a differential signal without the offset; we've chosen the resistor ratios for zero (differential) output at mid-scale (+50°C), so the differential signal goes from −80 mV to +80 mV. Note the 4-wire ("Kelvin") connection, to eliminate effects of cable and contact resistance. Now we need some gain.

B. The frontend amplifier

Microcontrollers come in such variety that you can usually find one with just the features you want. The ADuC800-series and ADuC7000-series of "analog microcontrollers" from Analog Devices is aimed at sensitive conversion applications, and this particular part has just the combination needed here: a true differential input stage, followed by a programmable gain low-noise amplifier and 16-bit ADC[27] that can be operated in a precision "chopping" mode (analogous to chopper op-amps) for low dc offset and drift. The datasheet includes tables of converter accuracy versus conversion rate, showing 15 bits of resolution for the ±80 mV range at 50 conversions/sec in chopping mode; that corresponds to 6 millidegrees, plenty good enough for this job.

Without such a convenient microcontroller (which Analog Devices calls a "microconverter"), the alternative is to use a differential instrumentation amplifier, preferably one

[24] A good place to find these things in the impressive *Temperature Handbook* from Omega Engineering.

[25] Commercial temperature controllers intended for use in systems with large thermal mass and slow response typically perform their PWM at very low frequencies, in the Hz region or slower, allowing the use of a relay rather than an electronic switch output.

[26] Accurate tabulations of Pt-RTD resistance list nominal values of 80.31 Ω and 157.33 Ω at these temperatures, typically accurate to a few tenths of an ohm.

[27] The ADuC848 has a 16-bit ADC; the similar ADuC847 has a 24-bit ADC, and the ADuC845 has two 24-bit ADCs.

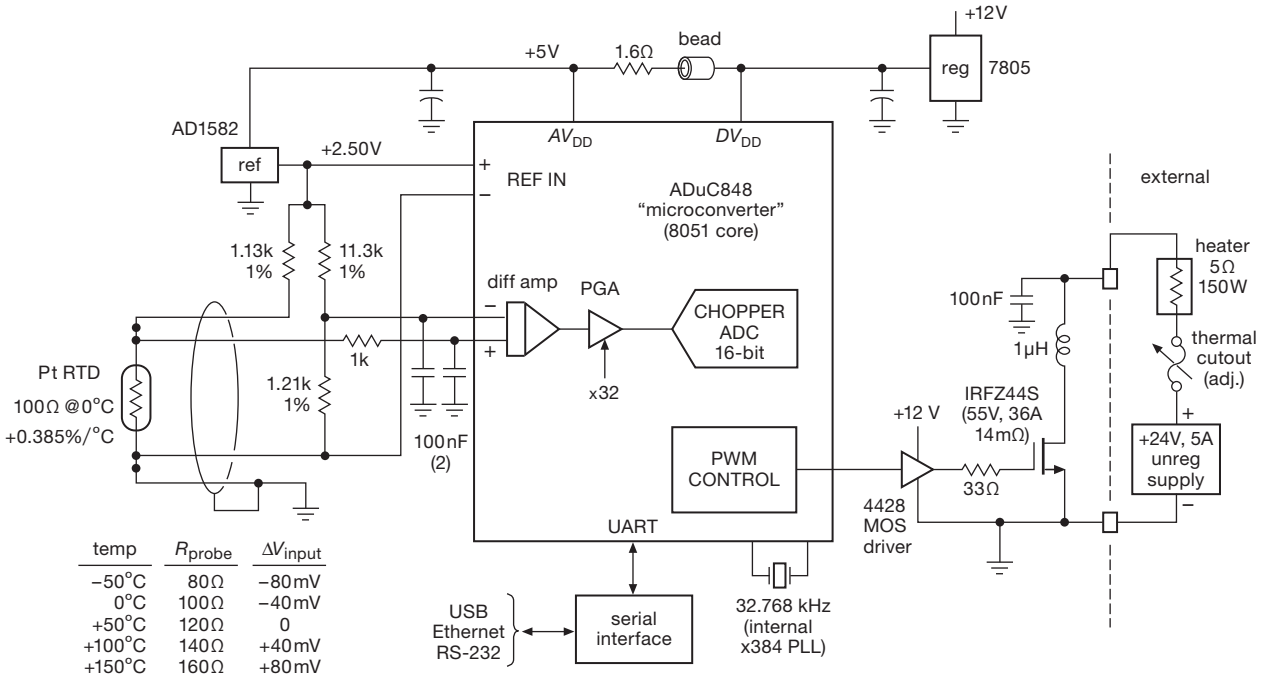


Figure 15.10. Temperature controller, with platinum RTD sensor and linear PWM heater. The ADuC848 "microconverter" includes a 16-bit ADC and programmable gain differential amplifier, aimed at analog applications requiring accuracy and low noise. The resettable thermal cutout is a good safety feature in the event of hardware or firmware failure.

with input common-mode range to its negative rail so it can be run from a single +5 V supply. Most instrumentation amplifiers (Table 5.8) don't have that feature; but a suitable candidate is the AD623 (Analog Devices strikes again!), which also has rail-to-rail output swing and can run from a single +3 V to +12 V supply.

Figure 15.11 shows how you would use it here, to drive the analog inputs of a more generic microcontroller: the output reference pin is tied to the midpoint (+1.25 V) of the desired conversion range, so that the differential input signal range of ±80 mV is mapped to the 0 to +2.5 V range of a generic single-ended ADC. (We could instead ground the reference pin, and replace the 1.21 kΩ resistor in Figure 15.10 with 1.0 kΩ, thereby balancing the bridge at the low temperature end.) For a situation like this, where you need accurate amplification of low-level signals, you can generally expect better performance with a dedicated instrumentation preamplifier, as compared with a bare microcontroller. However, the ADC in the particular microcontroller we chose is exceptional, and in fact has better accuracy and drift specifications than the outboard preamplifier: it specifies typical voltage offset and drift (in chopping mode) of 3 μV and 0.01 μV/°C, compared with the instrumentation amplifier's corresponding values of 25 μV and 0.1 μV/°C.[28]

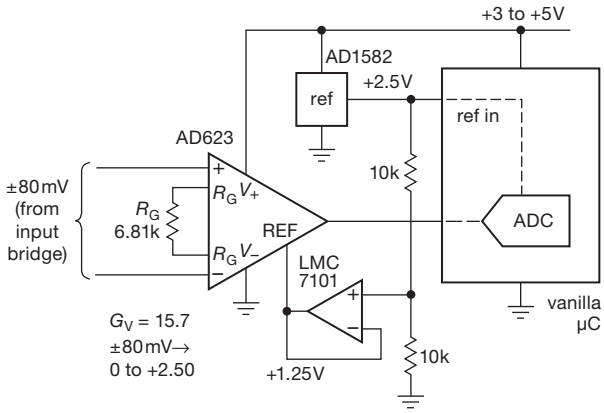


Figure 15.11. As an alternative, an outboard differential "instrumentation amplifier" could be used to map a small differential signal range (here ±80 mV) to the single-ended input range (here 0 to +2.5 V) of a generic microcontroller. In such an implementation, the gain-setting resistor sets the span, and the "reference" input pin sets the output to midscale when the input is balanced.

[28] In fairness it should be noted that the chopping process, which is re-

C. Clock and power supplies

The ADuC848 asks for a 32.768 kHz external crystal, with its on-chip PLL ×384 clock multiplier generating the primary core clock of 12.58 MHz (the controller has the requisite 12 pF on-chip capacitors, so the only external component is the crystal itself). The chip comes in +3 V and +5 V versions, with internal brownout detection and power-on reset, along with a watchdog timer. We chose the 5 V version, with a +2.50 V ADC reference. The 3-terminal reference (AD1582) need not be of great precision or stability, because the input circuit is ratiometric: the ADC's digital output is in fact independent of the reference voltage. The *RL* network decouples digital noise from the analog supply voltage, as recommended in the datasheet.

D. Pulse-width modulation

As we remarked earlier, this PID control loop operates with quasi-linear control of the heater, achieved by rapidly switching the heater full ON and OFF while adjusting the duty cycle (fraction of a cycle during which it is ON); that's called PWM (pulse-width modulation). That way we get the benefits of linear control, along with the efficiency of saturated switching. For an application like this, with its long thermal time constants, the heater won't know the difference between PWM and honest linear voltage drive.

In a purely analog world, you'd create the PWM switch drive signal with an analog comparator: one input gets a fixed-frequency sawtooth, the other gets a more slowly changing "feedback voltage" (indicating the need for more or less average output) that sets the duty cycle (Figure 15.12A). The latter might be simply proportional to the error, or in a more complicated system it might come from an analog PID like that in Figure 15.14. Here we use the microcontroller to do the same job *numerically*, by comparing a counter's incrementing value (the sawtooth) with a second number (the feedback) that is within the counter's range. In a μC without built-in PWM support, you would have to do this in software; see Exercise 15.3. The ADuC848, however, includes hardware to make PWM a breeze: you initially set the counter's range (maximum count) by loading a 16-bit unsigned number into a terminal count register (called PWM1); the incrementing count is continually compared with the (slowly changing) 16-bit feedback value resident in another register (called PWM0), creating a HIGH PWM output on a port pin (called P2.5) when the PWM count is less than the feedback (Figure 15.12B).[29]

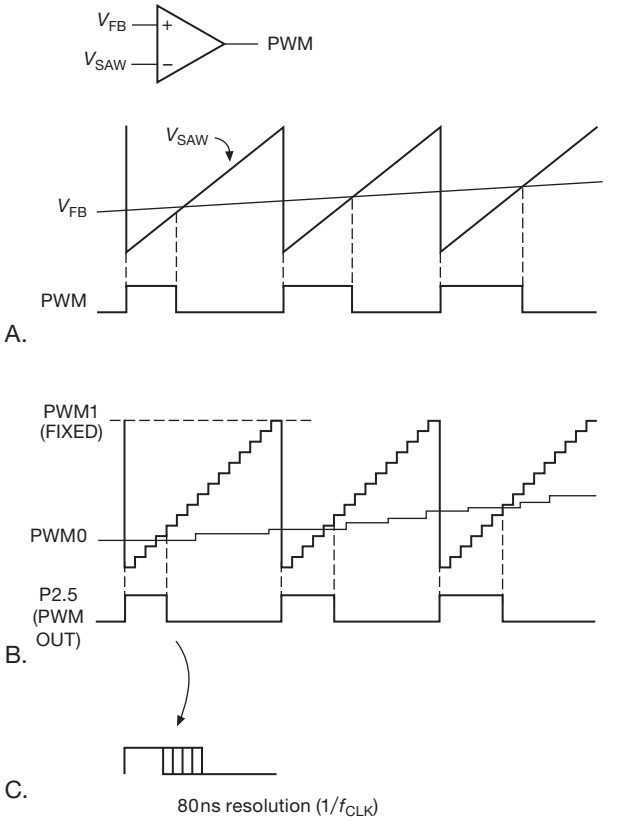


Figure 15.12. Pulse-width modulation (PWM). A. A slowly varying feedback signal V_{FB} is compared with a fixed-frequency sawtooth to produce the logic-level PWM output. B. Digital PWM substitutes a clocked counter and magnitude comparator. C. The digital PWM output has a granularity of $1/f_{CLK}$.

For this application we chose a maximum count of 1258_{10} (by loading hex 04EAh into internal register PWM1), to produce a PWM frequency of 10 kHz (the 12.58 MHz clock, divided by 1258). The PWM output pulses thus come at a 10 kHz rate, with widths corresponding to an integer number of 12.58 MHz clock pulses; that is, they are quantized with a step size of 1/12.58 MHz, or 80 ns (Figure 15.12C).

Exercise 15.3. Problem for a rainy Sunday: imagine you've already used the ADuC848's two PWM pins, and you need to create two more 8-bit software PWMs. You're given the register values, and during software PWM operation you have full control of the

sponsible for the excellent accuracy and stability, reduces the bandwidth and conversion rate. For our thermal-control application, with time constants of order 250 ms, speed is not a problem.

[29] This "Single Variable Resolution PWM" mode is one of six modes offered by this elegant chip. Such flexibility is unusual in your average microcontroller.

μC. Devise workable code, and see how fast the PWMs can run. The "Optimized Single-Cycle 8051 Instruction Set" and timings are given in the datasheet (page 20 of the current revision). The ADuC848 has a 12.58 MHz clock.

E. Output circuit

From a microcontroller output port you can't switch a load that is powered from 24 volts. And you *really* can't switch 5 amps! So we need a transistor switch, here implemented with a modest-sized power MOSFET. And, to drive its gate fully (to +12 V) and fast, we added a gate driver chip.[30] The 4428 is an industry-standard MOSFET driver, good for converting a logic-level input swing to a full-voltage swing at the gate; this inexpensive part (about $1) has dual outputs (inverting and noninverting), good for 1.5 A of source or sink into the capacitive gate load. The particular transistor we chose (the IRFZ44, rated 55 V, 36 A, TO-220 power package) is inexpensive (~$2), with R_{ON} sufficiently low (14 mΩ max, at V_{GS}=10 V) that we don't need a heatsink. A full 12 V gate drive minimizes the on-resistance, and the 1.5 A gate drive capability ensures fast gate rise and fall times, reducing MOSFET "class-A" dissipation[31] during switching.

A switching MOSFET with healthy gate drive produces rapid (nanosecond-scale) drain transitions, potentially generating substantial electromagnetic interference. The small output inductor and shunt capacitor form a lowpass filter of characteristic frequency 0.5 MHz, limiting output slew rate. This eco-friendly filter has an interesting side effect, however; namely, the production of an inductive positive-going voltage transient at each switch-off (Figure 15.13). Unless clamped, the inductive spike causes avalanche breakdown in the MOSFET. That sounds bad, but in fact power MOSFETs are tolerant of such abuse, with a specified repetitive avalanche energy rating limited primarily by heating effects (see the section on Pulse Energy in Power MOSFETs in Chapter *3x*, and the section on Transient Thermal Response in Chapter *9x*). For this circuit the periodic delivery of the $\frac{1}{2}LI^2$ energy stored in the inductor merely adds some allowable increment of MOSFET average power dissipation.

In total, then, there are three contributions to MOSFET heating: (a) I^2R_{ON} dissipation during conduction; (b) I_DV_{DS} class-A dissipation during switching; and (c) $\frac{1}{2}LI^2f_{osc}$ repetitive avalanche dissipation at switch turn-off. The following exercise explores these quantitatively for this circuit example.

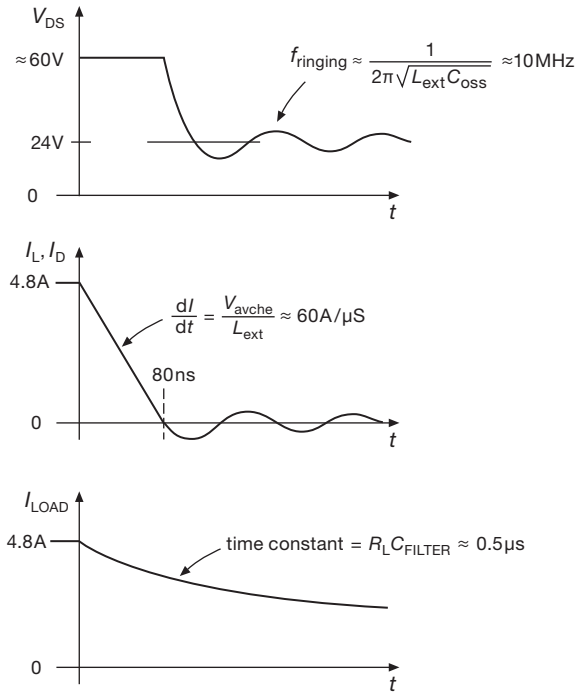


Figure 15.13. The output *LC* filter in the circuit of Figure 15.10 smooths voltage and current transients to the load (bottom graph); but the series inductor causes MOSFET avalanche breakdown at turn-off (top graph), during which the inductor current ramps to zero.

Exercise 15.4. Let's flesh out this last statement, in several steps.[32]

(a) What is the MOSFET power dissipation when it is ON, assuming R_{ON}=14 mΩ?

(b) Now calculate the average power dissipation contributed by class-A conduction during transitions, assuming the PWM is running at 10 kHz. For this calculation use a gate-to-drain ("Miller") charge of Q_{GD}=15 nC, and assume for the moment that we have been foolish enough to drive the gate directly from a logic output that is able to source and sink just 10 mA (which is considerably better than this μC's specs: a feeble 1.6 mA at 0.4 V sinking, and even feebler 80 μA at +2.4 V sourcing). You can assume a linear ramp of voltage and current, but to do the calculation properly you'll have to integrate the product $V_{DS}I_D$ over the duration of the ramp (and don't forget that there are two ramps per cycle).[33]

[30] See §3.5.4 and Tables 3.4b and 3.8 on pages 189–191 and 218 respectively.

[31] That is, the power dissipated during the transition times when the transistor is neither fully ON or OFF, and thus has both a substantial drain current and a substantial drain-source voltage drop.

[32] Refer back to §9.7.2 if you are unsure how to go about these calculations.

[33] Spoiler (if you want to skip the calculus): $E = V_PI_PT_{ramp}/6$ per ramp.

You should have found that the overall power dissipation is dominated by the class-A contribution during switching; in particular, the latter is nearly twice as much as the ON dissipation (580 mW versus 320 mW).
(c) The gate driver dissipates some average power, also; calculate it.
(d) Recalculate the average switching loss, assuming now a gate drive current of 1 A.
(e) Now calculate the contribution of inductor avalanche energy to the overall MOSFET power dissipation (we hope you'll agree that it is 115 mW).
(f) Check that the single-pulse avalanche energy is well below the datasheet's specification of 86 mJ (max).[34]
(g) Finally, under the assumption that the 1.5 A MOSFET gate driver has made the class-A dissipation negligible (compared with the conduction and avalanche losses), calculate the junction temperature rise above ambient, using the specified thermal resistance $R_{\Theta JA}$=40°C/W for this surface-mount MOSFET on 6 cm^2 of copper pad. If you (and we) have done the calculations correctly, the answer should quell any residual anxiety. Note also that the actual average R_{ON} conduction dissipation will be less than calculated, because the duty cycle (fraction of time ON) had better be less than 100%, or we're in serious trouble!

This exercise reveals some of the essential engineering process of tradeoff and iteration that goes into even a simple-looking output circuit like this.
(a) We chose a MOSFET with low enough R_{ON} to permit surface mounting without a heatsink, if desired (the IRFZ44S version: 40°C/W, good for ∼1 W dissipation).
(b) Budgeting about 1/3 W for conduction loss sets the maximum load current at about 5 A.
(c) Then we chose the heater supply voltage to produce 100+ W, to get things heated quickly.
(d) We added the *LC* filter to suppress RFI above ∼1 MHz.
(e) Then we confirmed that the avalanche power dissipated in the MOSFET from periodic dumping of the inductor's stored energy added only a modest additional dissipation, and stayed well within the ratings.
(f) Finally, we added a gate driver IC to minimize class-A conduction losses, which otherwise would dominate. During this process we made numerous adjustments to the heater supply voltage (+12 V, +24 V), heater current (2.5 A, 5 A), and inductor size (1μH, 5μH, 10μH), and toyed with the idea of using a MOSFET with lower R_{ON}, or even a TO-220 package MOSFET with a finned heatsink.

In the end we settled on the design shown in Figure 15.10 as a good overall compromise. However, it would be perfectly reasonable to choose a larger filter inductor, or run at higher current, or both, in which case you'd use a MOSFET with lower R_{ON}, perhaps in a larger package attached to a heatsink. The glib presentation of finished designs often conceals this kind of circuit design thinking.

15.6.2 The control loop

The microcontroller firmware's job is to implement a control loop, to maintain the bath at the desired "setpoint" temperature. Although this looks like plain ordinary feedback, of the sort used for an op-amp circuit, in reality it is bedeviled by thermal time delays; providing sufficient loop gain to stabilize the bath will invariably lead to thermal oscillations. This is a common problem in industrial control systems. As Jim Williams wrote,[35] "The unfortunate relationship between servo systems and oscillators is very apparent in thermal control systems."

The usual solution is what's known as a PID controller, in which ordinary proportional negative feedback (*P*) is augmented by negative feedback proportional to the rate of change of the error ("derivative," *D*), and also by a negative feedback term that grows with time according to the error ("integral," *I*). This can be done simply with op-amps, as shown in Figure 15.14.

The PID controller has to be "tuned" to the properties of the controlled system, to optimize the gains of the three feedback terms. An empirical procedure that works pretty well is the following: (a) with *I* and *D* terms turned off, increase the *P* gain until the system begins to oscillate, then back off a bit; the system will now exhibit overshoot and ringing, but not sustained oscillation, in response to a step change in setpoint; (b) now add *D* gain until the response to a step is critically damped;[36] (c) finally, while watching the error signal itself, add *I* gain to achieve minimum settling time.[37]

[34] As general guidance, if there's less than a factor-of-20 safety margin, you need to check the pulsed power dissipation using the MOSFET's specified "Transient Thermal Impedance." That's because datasheets use a particular set of operating conditions for their single-pulse avalanche spec; see §*3x.13*.

[35] In "Thermal techniques in measurement and control circuitry," Application Note 5, Linear Technology Corporation, December 1984.

[36] In the language of poles and zeros, you've brought in a zero to cancel the natural pole of lowest frequency in the physical system.

[37] Another way to tune a PID, which is due to Ziegler and Nichols (1942), uses the gain setting and frequency of oscillation in step (a) to determine, without further experimentation, the optimum *D* and *I* gains. We like the treatment in Tietze and Schenk's excellent *Electronic Circuits* (Springer, 2007).

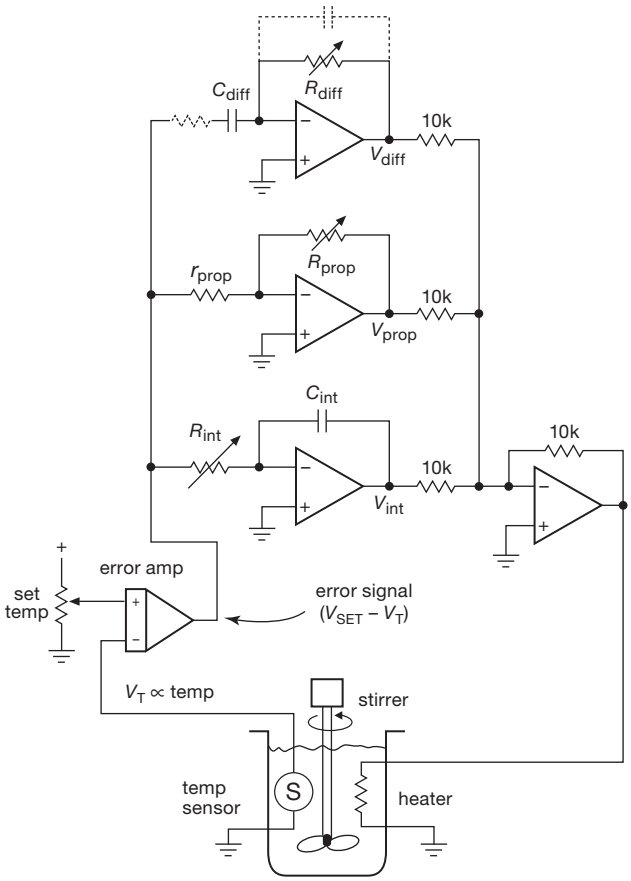


Figure 15.14. An analog PID control loop. The error signal (proportional to the difference between desired and actual temperature) drives an amplifier ("proportional"), an integrator, and a differentiator, with individual settable gains. Their combined outputs form the control signal to the heater. The dashed R and C are for differentiator stability. (You sometimes see the differentiator input taken separately from the upstream side of the error amplifier.)

15.6.3 Microcontroller code

The firmware to implement a PID in a microcontroller begins with the usual setups: ports, converters, timers, and communications. The interesting part is the main loop, where the measured temperature is digitized at equal time intervals (call the nth measurement T_n), and the results are used to compute numerically the P, I, and D outputs from the error "signal" $T_{error} = T_{set} - T_n$. The listing in Pseudocode 15.4 shows the simplest computation, assuming known tuning coefficients k_P, k_I, and k_D. In practice you would probably do some smoothing or filtering for the derivative term, which is prone to noise. You might also set up the ADC directly to perform periodic conversions, with polling or interrupts to signal the main loop's PID computation.

Pseudocode 15.4 PID main loop pseudocode

Initialize
zero the integration accumulator: $I = 0$
Main Loop
reset timer for 10 ms timeout
read temp, compute nth $T_{error} = T_{set} - T_n$
compute individual PID power output terms:
$P = k_P T_{error}$
$I = I + k_I T_{error}$
$D = k_D(T_{error} - T_{error-1})$
$T_{error-1} = T_{error}$
combine and update PWM:
PWM $= P + I + D$
wait until timer times out
repeat **Main Loop**

A. Some comments (algorithm)

The PID control loop, though of great popularity, is not the only game in town. In particular, there are nonlinear algorithms that, though difficult to treat mathematically, seem to work just fine. An interesting example is Steve Woodward's "take-back-half" (TBH) algorithm,[38] which has the nice feature of "one knob" tuning: you don't have to know anything about the "plant."

The control algorithm has two parts. It begins with a pure I-loop (controller output proportional to the integral of the error), which has the advantages of simplicity (only one "tuning" knob: the integrator gain), and of zero averaged error. However, the bad news is that the controlled variable oscillates about the target setpoint forever. Woodward's fix is to make a step correction at each zero crossing of the error, replacing the controller's current output with the average of current value and the value at the last zero crossing. We were curious how this would look, so we dragooned a student skillful in Mathematica (that's him in Figure 15.25 on page 1094) into running a numerical simulation of a TBH temperature controller. You can see the results in Figure 15.15, where we plot the controller output (heater power) and the response of the controlled system (sensor temperature). We started it up at ambient temperature (20°C), teasing out its transient behavior by switching the setpoint between the two values shown. For this simulation we used heater and sensor time constants of 0.5 s, a delay time (from fluid flow) of 0.1 s, and a thermal

[38] "Temperature controller has "take-back-half" convergence algorithm," *EDN*, page 90, September 15, 2005.

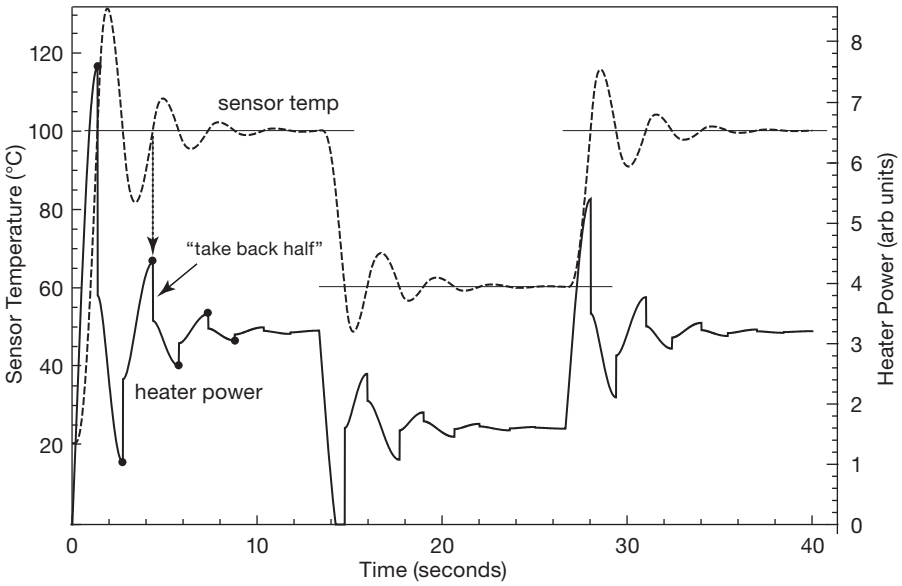


Figure 15.15. A nonlinear control algorithm: Steve Woodward's "take-back-half." This numerical simulation shows the behavior for a sequence of step changes in target temperature (alternating 60°C and 100°C). The controller is a pure integrator, but responds at each moment of zero temperature error (the black dots) by resetting its output (heater power) to the average of its current value and the value at the previous reset.

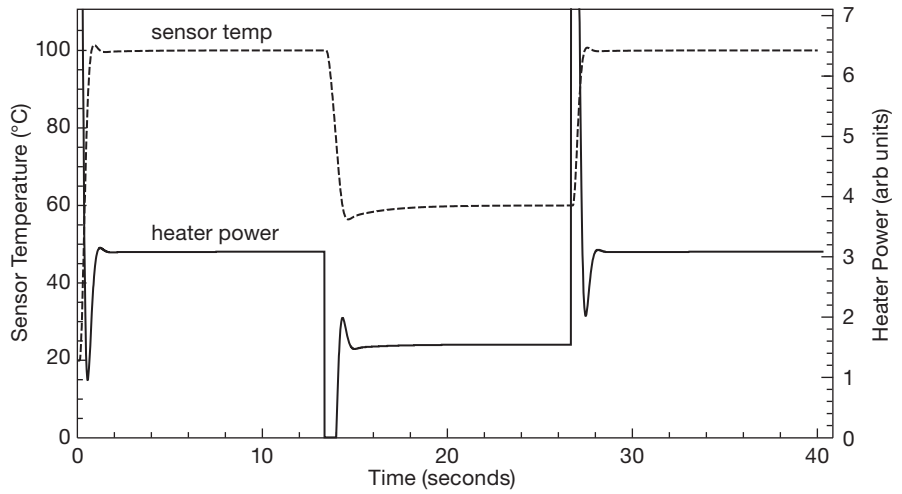


Figure 15.16. Numerical simulation of the system of Figure 15.15, this time with a classic tuned PID controller.

relaxation time to ambient temperature of 0.5 s. The one "knob" (integrator gain) was set to produce a reasonable convergence.

Having set up the TBH simulation, we couldn't resist running a classic PID. After fooling around with the tuning (*three* knobs, this time), we got the results shown in Figure 15.16. The PID does a considerably better job than TBH; but it requires some skill in tuning to find the *P*, *I*, and *D* coefficients suited to the specific parameters of each controlled system.

B. Some comments (Hardware):

This example illustrates how a particularly good choice of microcontroller can greatly simplify the hardware (and coding): this microcontroller combines a good ADC (real 16-bit performance), with the low offset and drift you get

with chopping, and an internal differential amplifier of programmable gain, so that the low-level RTD input can be used directly. The differential reference input makes it easy to rig up true ratiometric conversion, insensitive to variations of the reference voltage source. And internal PWM hardware dramatically eases the coding burden.

We've kept the block diagram lean, so as not to distract from the essentials of the analog input circuitry, and the PWM output drive circuitry. But it's easy enough to add some of the creature features shown in previous microcontroller examples: an LCD display of setpoint and actual temperatures; a keypad or thumbwheel entry for parameters such as a simple setpoint, or a more complex time/temperature profile; indicator LEDs and alarm buzzers for fault conditions, and the like.

15.7 Design example 5: stabilized mechanical platform

The final microcontroller example is fun – a motor-driven stabilized two-wheel contraption (which its inventor calls a *Psegué*), shown in action in Figure 15.17.

This thing is a tricycle without its front wheel, therefore unstable without active feedback. The prototype is, of course, Dean Kamen's "Segway™ Personal Transporter" that delighted the world at its debut in 2001. Weird and wonderful homemade variants are sprouting up, propelled by the enthusiasm of the lively hobbyist–nerd community. One such hobbyist is the young man who lives across the street, who asked if he could hang out in our lab to try out his ideas for making a stabilized motorized platform. We doubted he would succeed; we were wrong.

Figure 15.18 shows the system he finally arrived at (after some amusing mishaps). It is a digital PID control loop, implemented in an NXP ARM7 microcontroller. The latter comes nicely packaged on a 6 cm square board, which includes voltage regulators, connectors for analog, digital, and serial ports, and LCD display; it is called the MINIMAX/ARM-C, sold by BiPOM Electronics for $100.

To sense the platform's instantaneous angle, Jesse used a two-axis solid-state accelerometer, rotated 45° to the vertical, feeding a difference amplifier. The output voltage is proportional to the sine of the platform angle, passing through zero when the platform is horizontal. This provides the analog input for the proportional and integral terms of the PID loop; for the derivative term he used a gyro, whose output is directly proportional to the rate of change of tilt. You steer this thing by pushing the vertical post sideways, which squeezes a pair of resistive force sensors under the post's base. The microcontroller has a Bluetooth link, for playing around with the loop parameters while riding. Figure 15.19 shows the hardware.



Figure 15.17. Stabilized two-wheel contraption, demonstrated by its creator (Jesse Colman-McGill).

On the output side the microcontroller drives a pair of H-bridge dc motor drivers with PWM logic outputs, as commanded by the PID loop. The dc permanent-magnet gearmotors are substantial beasts, able to put out nearly a horsepower.

The PID loop is paced by one of the microcontroller's internal timers, operating at a 100 Hz heartbeat. The basic PID controller is augmented by some gimmicks that were arrived at by experiment, for example, an extra boost to get the thing moving from a dead stop, and some modifications to the PID gains according to the measured load. A fancy name for such unauthorized patches to the basic PID is *heuristics*; whatever you call them, they're needed, and with some luck they can work well. See Pseudocode 15.5.

Pseudocode 15.5 Psegué main loop pseudocode

```
Main Loop
    reset timer for 10 ms timeout
    read sensors
        tilt-sensing accelerometers (2)
        rotation-sensing gyro
        steering force sensors (2)
    compute PID with speed-dependent parameters
    apply heuristic rules
        dead-zone correction
        threshold boost
        load-dependent PID gain multiplier
    send updated torque command to motors
    if (logging) increment log_loop_count
        if (log_loop_count=10) log data & clear log_loop_count
    TimerCheck: if (timer not expired)
        if (command byte from wireless input FIFO buffer)
            append to line buffer
                if (newline) parse & execute
        write logging byte to wireless output FIFO buffer
    repeat TimerCheck
    repeat Main Loop
```

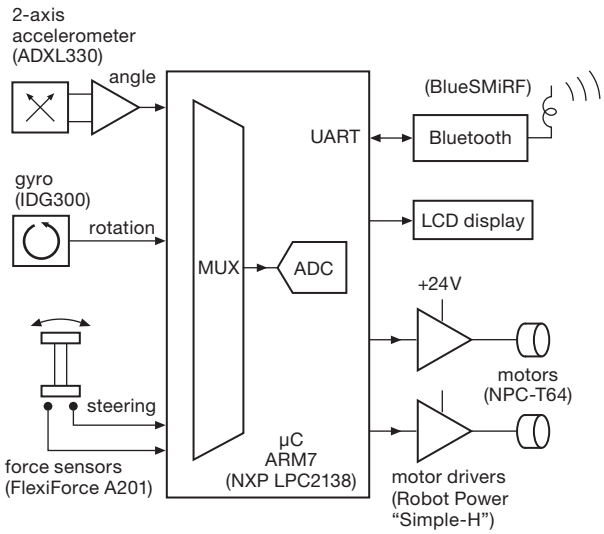


Figure 15.18. Block diagram of the stabilized scooter. The use of prefabricated hobbyist modules (gyro, accelerometer, wireless, microcontroller board, and motor drivers) simplified construction and wiring. The actual part numbers are in parentheses.

15.8 Peripheral ICs for microcontrollers

Our experience in electronics engineering comes from the real world, where an enormous variety of measurements and interactive control are needed. Having a programmable microcontroller is good, and having lots of built-in interfacing circuitry is great – but the world is bigger than that. When we talk about the rubber meeting the road, what we need is a tire, a wheel, and other specialized devices that aren't included inside a microcontroller.

In this section we've created three drawings that show sixty examples of specialized interface devices, complete with sample part numbers. It's our hope that you'll point your browser at Octopart, etc.,[39] and read the datasheets for some of these parts, as inspiration for searching out additional examples and alternative parts.

DACs and ADCs are a common peripheral for interact-

[39] Octopart is an excellent parts locator, showing availability and pricing of components at stocking distributors; the site also includes datasheets for most parts. An especially valuable feature is the ability to see the status of parts, and their activity at various distributors. If a part has been discontinued, you'll still find out what it was once possible to make (and what its specs were), and you can hope someone else will have created a similar part. *One caution*: as of this writing, Octopart only lists components that are carried by distributors, and not those sold directly from the factory. If Octopart doesn't show a part, that doesn't mean you can't get it. Sometimes the factory has to be your source, even for mainstream manufacturers who also sell through distributors. We have found that in most cases the manufacturers make it easy to buy directly, even in small quantities.

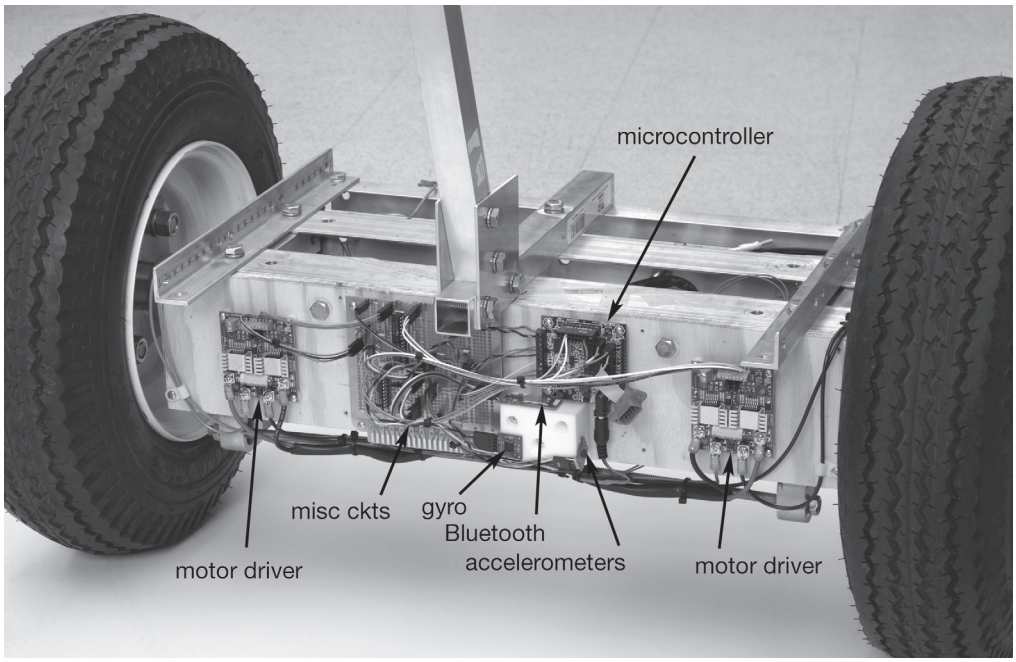


Figure 15.19. Closeup of the sensors and electronics. The difference signal from the accelerometer pair (each inclined 45° to the vertical) is a measure of fore–aft tilt angle, and the gyro provides a direct measure of the time derivative of tilt. The gel-cell batteries live in the space behind the electronics.

ing with real world stuff; if that's all you're interested in at the moment, use Tables 13.2, 13.3 and 13.11 (DACs), and Tables 13.5, 13.6 and 13.9 (ADCs). These tables include columns indicating SPI, I^2C, or other interface methods.

When designing a gadget with an embedded microcontroller, there are lots of "devil in the details." We could prattle on for pages about how to connect to many wonderful peripheral devices. Instead we present a guided tour and navigation aid, in the form of Figures 15.20–15.22, tagged with numbers to match the brief outlines that follow. We've put plenty of cross references to relevant discussions elsewhere in the book, along with some selected part numbers. We've broken it down into several figures, because there's so much stuff that you can attach to a microcontroller: devices that can connect directly to the μC (Figure 15.20); devices that connect to an SPI bus or its near relatives (Figure 15.21); and devices that connect to an I^2C bus (Figure 15.22). Your programming quality-of-life may be improved if your microcontroller includes a built-in interface for your chosen bus, like the one in Figure 10.86.

15.8.1 Peripherals with direct connection

Figure 15.20 shows a microcontroller, heavily encrusted with devices that connect easily to standard "internal peripherals" such as digital I/O port pins, serial communication ports (UART, USB, Ethernet), ADCs and DACs, PWM outputs, and the like. It also shows important support chips such as power control, watchdog, and external oscillator. In outline form we've collected some explanatory comments, useful part numbers, and references to relevant discussions elsewhere in the book.

A. Outline for Figure 15.20

1. Power-source selector. If the system runs from battery power, an ICL7673 with two *p*-channel MOSFETs makes a nice power selector, acting as an ideal diode-OR, automatically connecting to the higher of the two voltages. For rechargeable battery management see *§9x.2*.

2. Power supervisor, power-ON-reset, watchdog. Brownout protection, don't let the μC do anything important if its supply voltage is too low; reset it. See §10.8.1B, and Table 10.6 on page 756. There's a huge variety of choices available. Take a look at the ADM705 and TPS3306 supervisory ICs (which include a watchdog) and the ADM691 series (which also incorporate a battery-backup-switch feature).

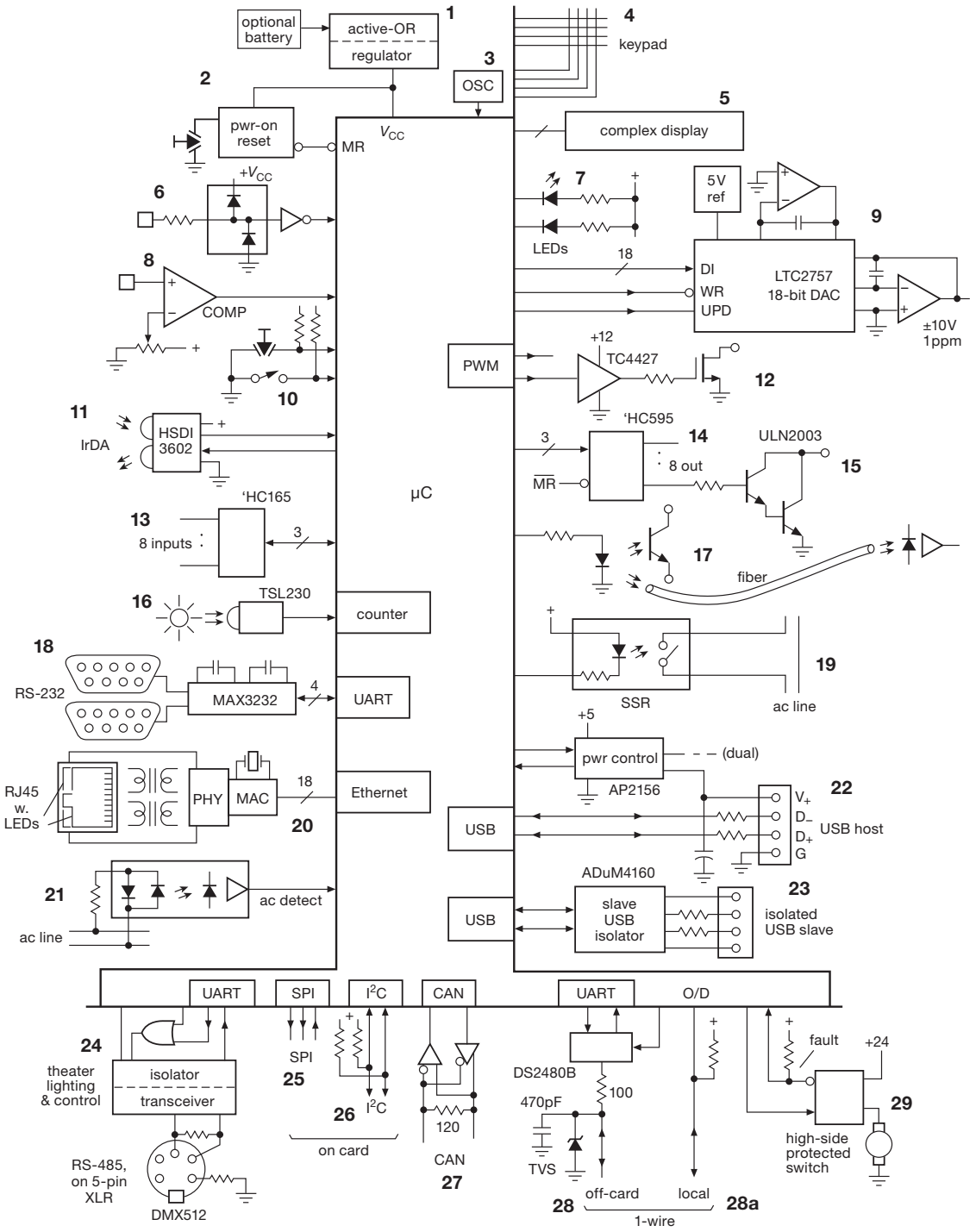


Figure 15.20. An assortment of peripheral devices that can be connected to a well-endowed microcontroller. Some of the devices require specialized interface circuits within the controller. The interface items marked with bold numbers **1** to **29** refer to descriptive paragraphs in §15.8.1.

3. μP oscillator. Internal, see §§15.2.1E and 15.9.3. External, see §7.1.6 for choices and cautions.

4. Keypad scanning techniques. See Figure 15.8.

5. Panel-display choices. LED, LCD, CFD, tricks. See §§10.6.2, 12.5.3, 14.6.

6. Logic inputs, external. Protect μC against ESD with separate user-repairable input gates, and the instrument's programmed μC survives; see §12.1.5.

7. LED and indicator outputs. See §§12.4, 12.4.5A, 12.5.1.

8. Analog threshold detection, discriminators. See §§12.1.7 and 12.3, and Table 12.2 on page 819.

9. DACs. See §§13.2 and 14.6.2, and Tables 13.2 on page 893 and 13.3 on page 894; fast 18-bit accurate parallel DAC LTC2757.

10. Switch and pushbutton inputs. See §12.1.4.

11. IrDA infra-red transceiver. HSDI-3602.

12. Power MOSFET switches. See §§3.5, 12.4, and 15.6, and Table 3.4b pages 189–191. MOSFET driver ICs, logic to 12 V MOSFET gate drive, see §3.5.3; classic TC442x family (see Figure 3.97 and Table 3.8 on page 218). For PWM see also §15.6.2 and §*9x.5*.

13. 8-bit parallel-in shift register. See Figure 12.40, suited for onboard use. 74HC165 (and other '165 logic ICs), \$0.35, compare with a simple 8-bit I^2C expander chip at ∼\$2.30.

14. 8-bit parallel-out shift register. See Figure 12.40, e.g., 74HC595 logic IC, \$0.32; also check out the 74HC594 and '567, which have a double buffer. For SR chips with built-in power drivers (e.g., TI's TPIC6C595), see §12.4 and Table 12.3.

15. Darlington and MOSFET driver ICs. See §12.4. Drive loads with the venerable ULN2003 Darlington 7-unit array, \$0.33, or with individual logic-level MOSFETs, see Table 3.4a, on page 188. Some other useful driver ICs include the ULN2803, SN75468, TPL7407, MC1413, ULN2068 and TD62783.

16. Light-intensity sensor. TAOS TSL230 provides a light-to-frequency signal, allowing for easy, accurate measurements over six decades of light level. See Figure 12.81.

17. Optical isolation. See §12.7 and the drawings and part numbers in Figures 12.85–12.88. Logic optocoupler, jellybean H11L1, HCPL-2201. For long-distance optical-fiber data links, with connectors, see §12.7 and Figure 12.98.

18. Serial data-communication interfaces. RS-232, with ±7 V levels; see §12.10.4. Use the classic DS14C88 and DS14C89 chips, or for simplified operation on +5 V, second-sourced MAX232, \$0.90 from TI; or 15 kV ESD-protected MAX3232E runs from 3.3 V or 5 V, \$3. These have 2 Tx and 2 Rx, thus one port plus two control lines, or two ports. For RS-485, see §§12.10.3, 14.7.8. A typical transceiver chip is the LTC1485.

19. ac power switch. Isolated SSR, zero-crossing triac or back-to-back SCRs: see §§12.7, 15.4, and Figures 12.91–12.93. Available as low-current ICs, e.g., Fairchild MOC3043, \$0.80, or as powerful modules with screw terminals.

20. Ethernet. Protocol processing, transformer coupled, see §14.7.16 and Figure 12.124. Easy-to-use chip, Silicon Labs CP2201, about \$4.50 in qty 100. You'll also need a transformer (e.g., Pulse PE-36023) and an RJ-45 connector. Most folks buy these combined into one compact part, complete with two indicator LEDs, e.g., Pulse J00-0065NL, \$4.75.

21. ac power detect. To monitor relay outputs, blown fuse, respond to ac-line signals. Opto-isolated; see §12.7.7 and Figure 12.94A, e.g., MOC256.

22. USB. Supported by a controller in the μC, see Figure 10.86. USB hosts need a power supervisor with a 500 mA CL, etc., e.g., AP2156 dual (see Table 12.5). USB clients, see §§14.7.13 and 15.9.2.

23. Isolated USB. Analog Devices' ADuM4160.

24. DMX512 theater lighting and control. 5-pin XLR connectors, to 1200 meters. 250 kbaud (UART in μC), with OR gate for the packet-framing "MAB" symbol. Isolator and RS-485 transceiver, MAX1480 or MAX3480B, see §12.10.3. Near and far ends terminated. Theater electricians: with luck you'll be handed proper 120 Ω twisted cable. DMX512 is being superseded by DALI.

25. SPI. Serial microprocessor to chip interface. It's an SPI jungle out there, with many different interface rules grouped under the SPI label; read the IC datasheet very carefully. SPI interfaces are often implemented as individual bit-bang lines, and, as such, SPI may be more of an interface scheme than a multiple-device bus. See also §§14.7.1 and 15.8.2 (the next section).

26. I^2C. Chip-to-chip interface; multi-master. Two-wires, open-drain wired-AND operation, with pullup resistors; see §§14.7.2 and 15.8.3. Maximum bus capacitance 400 pF. Unlike SPI, the I^2C bus is well specified (and its use tightly controlled) for predictable results.[40]

27. CAN-bus. Two-wire bidirectional bus, differential signaling; see §14.7.15. Generally a CAN-bus

[40] The I2C-Bus Specification, Version 2.1, January 2000 (NXP Semiconductors).

transceiver is required, e.g., ON Semi's AMIS-42673 or AMIS-41683, and (if not built-in) an SPI-based controller like the Microchip MCP2515.

28. 1-wire bus. Power and bidirectional data signaling on one wire; see §14.7.3. Championed by Dallas, now Maxim. UART transceiver DS2480B. USB to 1-Wire, DS9490.

28a. Alternative simple 1-wire for μC to local chips, open-drain I/O and pullup resistor.

29. Power switches. High-side to 60 V, 550 A, protected, with fault feedback; see §12.4.4 and Table 12.5; e.g., BTS432, IPS6031.

15.8.2 Peripherals with SPI connection

Figure 15.21 shows a variety of available peripherals that communicate via a simple SPI serial 3-wire master–slave protocol. The SPI bus[41] (§14.7.1) normally consists of three lines: a clock (SCLK); data in (SDI), and data out (SDO′). But there needs to be an additional chip-select line from the processor to each slave device; this is usually called CS′, but it's sometimes called other names like load (LD), or enable (EN), etc. This signal, by whatever name, generally transfers the contents of the device's serial-input shift register into an internal data register when CS′ is disasserted (brought HIGH) after the data bits have been shifted in. You can dedicate one μC pin for the CS′ of each device, or you can save a few pins by using a 74LVC138 (or other '138) decoder IC; see §10.3.3D. Furthermore, some peripheral chips on a bus may require additional individual connections to the microcontroller; for example, for interrupts, to reset the device, etc.

Signal-name usage in SPI may seem inconsistent; for example, the controller's data-in pin (SDI) is connected to the slave device's data-out pin (which might be called DO). Similarly, the master's SDO pin is connected to the slave's DI. A better scheme is to name your signals unambiguously as MISO (master-in, slave-out) and MOSI (master-out, slave-in); see §14.7.1. Sadly, you won't find these names on most device datasheets.

As we stated before (§14.7.1) the SPI "standard" is badly fragmented. For example, in some SPI slaves a single pin may be used for sending data in both directions. This requires the controller to reverse the direction of a pin, so it can receive data back on the same pin that it used for sending a command.[42]

A. Outline for Figure 15.21

30. Touchscreen. *XY* position digitizer, pressure sensitive; see §13.11.2 and Figure 13.73: AD7873.

31. Serial EEPROM. 8-pin package, §14.4.5B: 25LC080A, AT25080, M24C02 (256×8, $0.09 in 2500-pc reel)

32. Accelerometer. Three-axis MEMS: ADXL345, Freescale MMA7455L.

33. SD card. "SecureDigital": SD,[43] miniSD, and some microSD memory cards; §14.4.5C. These cards communicate via standard SPI, so this interface consists simply of the socket, with no additional electronics needed! The miniSD and microSD cards have 10-pin connectors. Pin 1 is CS′; but it's also "card detect," with a 50k pullup resistor, to let you know that a card is plugged in. You pull pin 1 low to initiate SPI communication mode.

34. "SparkFun" modules. Wide selection of easy-to-use modules with through-hole connections; e.g., three-axis accelerometer (ADXL335), two-axis gyro rate sensor (LPY503A), or three-axis magnetometer (MAG3110 or HMC5883). Also see Adafruit modules (adafruit.com), e.g., OLED graphic displays available in both SPI and I^2C interfaces.

35. Digital pot. DCP, EEPOT; accurate ratio (∼1%), but overall tolerance typically ∼20%; low voltage; see §3.4.3E; e.g., 10kΩ, 10-bit, 1024-level MAX5481, $2.60.

36. Digital resistor. Resistance tolerance 1%, operates to ±16 V; e.g., AD5292. See the digital resistor section in Chapter *3x*.

37. ADCs. See §§13.11, 14.6.2, 14.7 and Tables 13.6–13.12; e.g., LTC2412, AD7927 (Figure 13.74), AD7734.

38. Digital capacitor. Give range and step size; see the digital resistor and capacitor sections of Chapter *3x*; quartz FLEcap: MAX1474.

39. ac power monitor. Including power factor, see §13.11.1: ADE7753, with 8052 μC, ADE7769; needs an *i*Coupler isolator, ADuM3260.

40. DDS RF frequency synthesizer. See §§7.1.8, 7.1.9C, 13.13.6, and 6.2 (on *LC* filters); e.g., AD9954.[44]

---

[41] Analog Devices' AN-877 is a useful application note, describing the use of the SPI bus in various of their high-speed conversion devices.

[42] And of course that data appears on the MOSI pins of the other bus riders. It's harmless enough, because their chip-select pins are disasserted; but it's ugly.

[43] *SanDisk SD Card Product Family, OEM Product Manual*, Table 3-2, and *Physical Layer Simplified Specification*, Version 2.00, Chapter 7.

[44] Analog Devices is a giant in Direct Digital Synthesis ICs.

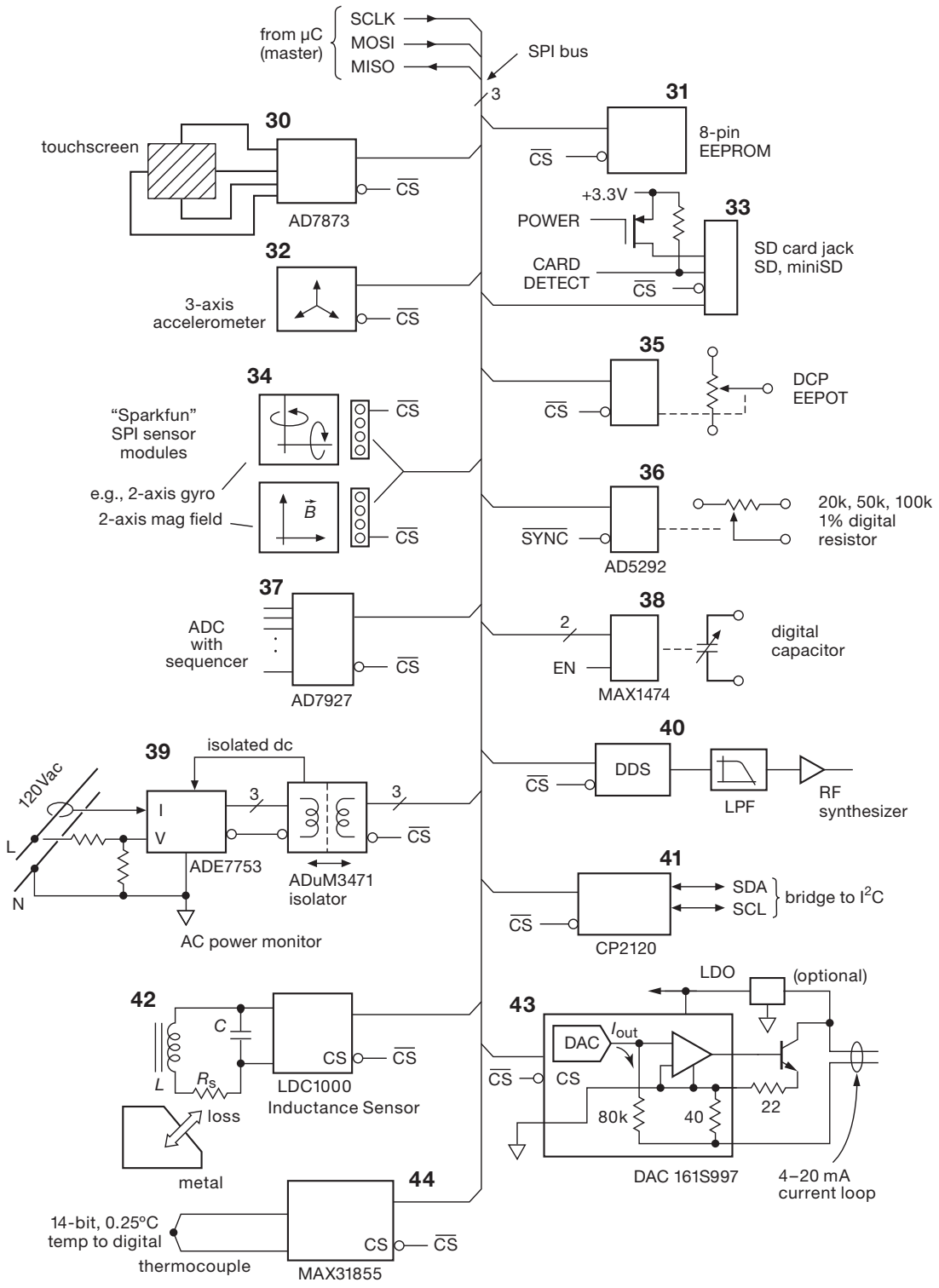


Figure 15.21. An assortment of SPI slave peripherals, well suited for microcontroller applications. The SPI interfaces marked with bold numbers **30** to **44** refer to descriptive paragraphs in §15.8.2.

41. SPI to I2C Bridge. Silicon Labs CP2120, Microchip MCP2515, NXP SC18IS600. See also next section, and §14.7.2.

42. Inductance sensor. See Table 13.12. TI introduced the first inductance proximity sensor IC, the LDC1000, which detects inductance and loss changes in an external *LC* resonant circuit. Operating frequency 5 kHz to 5 MHz.

43. 4–20 mA current loop. See §14.7.8 and Table 14.3. The DAC161S997 uses a 16-bit current-output DAC to program a precision current signal for conveying analog measurements in industrial environments.

44. Thermocouple ADC. The MAX31855 includes cold-junction compensation for seven types of thermocouples, and provides 0.25°C resolution from −270 to +1372°C (compare with Cirrus CS5532, suited for thermocouple use but lacking the valuable compensation, see Figure 13.67).

15.8.3 Peripherals with I²C connection

We introduced the I²C (sometimes written IIC) bus in §14.7.2, where its advantages and disadvantages were described in some detail. To summarize, I²C is a multimaster half-duplex "packet-oriented" bus, intended for chip-to-chip serial communication (I²C stands for inter-integrated circuit), with a well-defined addressing and data transfer protocol.[45] Compared with SPI's 3-wire "bus" plus individual chip-select lines, I²C is a true 2-wire bus. A data transfer goes like this. The master sends an initial byte containing a 7-bit address and a direction bit; the addressed slave responds with an acknowledgment bit, after which data bytes move from sender to recipient. The whole transaction is framed by unique START and STOP symbols.

I²C is a more complex protocol than SPI, and requires each device on the shared bus to have a unique 7-bit address. It's well suited for peripherals with lots of registers (you can send a register address as part of the packet), but its half-duplex nature and required addressing overhead make it less well suited to fast and continuous data streaming. In general, the choice is made by the manufacturer of the peripheral; most microcontrollers include support for both I²C and SPI.[46]

[45] You'll find statements like this in the datasheets of conforming I²C peripherals: "I²C is a trademark of Philips Corp. Purchase of I²C components from [manufacturer] conveys a license under the Philips I²C Patent Rights to use these components in an I²C system, provided that the system conforms to the I²C Standard Specification as defined by Philips."

[46] For controllers without I²C support, you can use an SPI-to-I²C bridge (as in Figure 15.21), driven either with the microcontroller's internal SPI hardware (if present) or with bit-banging via individual port pins. The bridge takes care of the fussy I²C timing and half-duplex signal reversals.

A. Outline for Figure 15.22

45. Parallel-bit ports ("GPIO" – general-purpose I/O). Definable wires, interrupt: 8 bits, STMPE801; 24 bits, STMPE2401; consider also TI's 16-bit TCA6416.

46. LED backlight. Smart backlight driver, fade, ambient-light responsive: ADP8860, ADP5501, with 4×4 keypad ADP5520.

47. Real-time clock (RTC). For example, NXP's PCA8565 (0.65 μA at 1.8–3.3 V, 32 kHz xtal, with alarm and timer interrupts), or the fancier PCF2129 (which includes internal xtal and TCXO, selectable SPI or I²C interface, and a nice time-stamp function to capture the time of an event even if your μC is sleeping) NXP's PCF8563 and Seiko's S-3590A are popular parts, and available for less than a dollar.

48. Clock synthesizer. See §13.13: ON-Semi FS714x, Silicon Labs Si5338 to 710 MHz, four LVDS out, with separate V_{CC} levels.

49. Three-axis accelerometer. MMA7660FC, $0.68, MMA7455L (I²C mode), or a variety of I²C modules from Adafruit, for example their MPL115A2 barometric pressure/temperature sensor, or L3GD20+LSM303+BMP180 10-DOF inertial measurement unit, $30. Many of the "shield" part numbers from AdaFruit and SparkFun match the IC part numbering of the mounted sensor ICs, which are available from distributors. The shields make it easy to experiment with these (generally tiny SMT) parts.

50. 8-pin EEPROM. See §14.4.5B: Microchip 24LC256 (32k×8, 2.5–5.5 V, low-cost $0.76), 24AA256 (1.8 V).

51. ADC subsystem. For example, AD7294: 12-bits, six inputs (3 temp, 2 current sense), four outputs, alarms, ($14), see Figure 14.41. The AD7730, intended for use with strain gauges, scales, etc., includes many functions (but uses the SPI interface), see Figure 13.75.

52. Stepping-motor position IC. For example, ON-Semi AMIS-30624 (both parallel and I²C control options): has position memory, a sine table for 1/16 microstepping, speed ramps, two H-bridges, V_S to 29 V, 800 mA, max.

53. Touchscreen. ST's STMPE811, $1.47.

54. Digitize video. NTSC and PAL composite, S-video, §14.6.2 and Appendix I; 8-bit TI's TVP5150, $4; 10-bit dual 30 Msps TVP5147, $5.

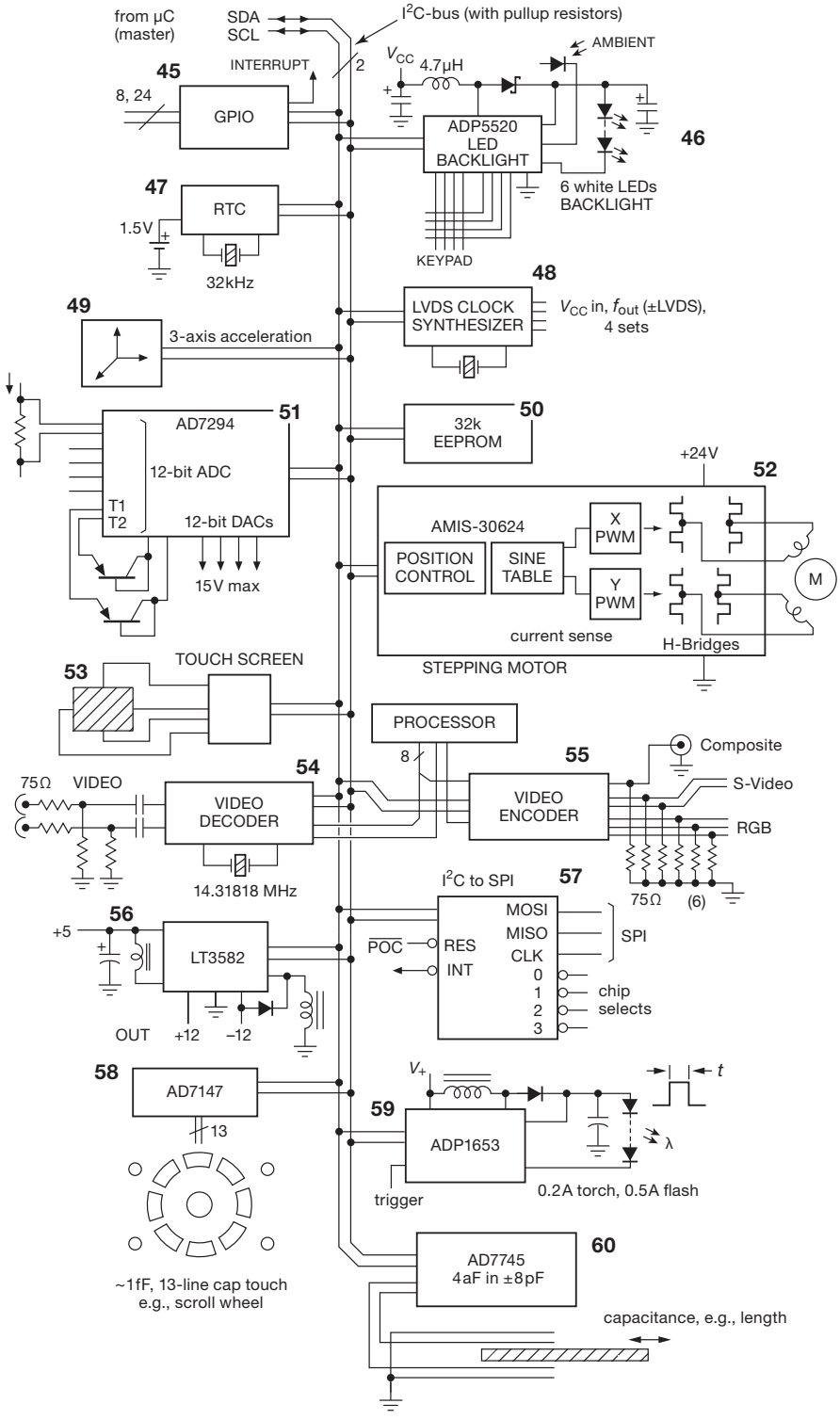


Figure 15.22. An assortment of I²C bus peripherals, well suited for microcontroller applications. The I²C interfaces marked with bold numbers **45** to **60** refer to descriptive paragraphs in §15.8.3.

55. Video generation. See §14.6.2: from pixel stream, Cirrus Logic CS4954; and for HD, Analog Devices ADV7390–93, see Figure 14.36.

56. Bipolar switching supply. See §9.6; Linear Technology's LT3582 is programmable in 25 mV steps to over ±12V (not all required parts are shown).

57. SPI controller. I^2C-bus to SPI bridge, see §§14.7 and 15.8.2. NXP's SC18IS602B, SPI master with four $\overline{CS}$ select lines.

58. Capacitance touch sensing. Analog Devices AD7147 provides 1 fF sensitivity on 13 input lines. Use to make touch pads of your own design, such as the scroll wheel shown. STMPE321 (three touch lines only), $1.30.

59. LED "torch" or flash. The ADP1653 boost converter (both parallel and I^2C options) has programmable current for a 200 mA torch (British for *flashlight*), or with timing for a 500 mA flash with trigger.

60. Precision capacitance sensor. Analog Devices AD7745 has a 24-bit converter and provides 4 aF resolution with a ±8 pF range, for precision position measurement, etc. The AD7746 has two channels. See Table 13.12.

15.8.4 Some important hardware constraints

Microcontroller systems present some unexpected circuit and timing constraints that you have to worry about. Here are a few that come to mind; they are discussed in some detail elsewhere in the book, as indicated by the section references.

A. Low supply voltages

Fast microcontroller core logic continues to shrink in feature size, with correspondingly lower supply voltages. And microcontrollers, like their more computationally adept micro*processor* cousins, present the dc supply with abrupt load-current steps, going from standby to full current on time scales of nanoseconds. You need efficient buck converters (or point-of-load voltage ratio converters) with low output impedance and good step response; and liberal use of SMT bypass capacitors. See §*9x.3*.

B. Logic-level translation

You're generally dealing with multiple supply voltages in an embedded application, with some incompatible logic-level requirements. See §12.1.3 for lots of detail.

C. Critical peripheral timing

Some "high-maintenance" external peripherals require great attention to timing. In the upcoming section on software, we illustrate this with the example of a high-throughput multichannel ADC system (§15.9.2), which stresses the timing capabilities of a modest-speed microcontroller.

D. Dual dc supply sources: battery and ac line power

Portable devices run from internal batteries, usually rechargeable by an ac adaptor. You need seamless switchover, battery-charging and protection functions, and the like. See §*9x.2*.

E. Reset supervisor

The desk telephone of one of the authors crashes (!) if the buttons are pressed too lightly or too fast. It gets wedged and has to be rebooted (by unplugging the power). That's a design flaw (we had three of the same model, with the same behavior), remedied by careful design with a watchdog and reset supervisor. This task is complicated by use of several supply voltages (which present their own problems, for example proper sequencing during power-up and turn-off). See §10.8.1B.

15.9 Development environment

The five design examples earlier give a glimpse of the enormous range of applications for which microcontrollers are so well suited. But these wonderful gadgets won't do anything until you've loaded a program into them. For that, you've got to (a) do the program coding, and (usually) simulation; (b) load the program into the target in-circuit microcontroller; and (c) check out and debug, if necessary (it will be!), the loaded code. The detailed landscape of the software and hardware tools to do this (the "development environment") is continually improving. Here we summarize the current situation, advising as always that the practitioner should stay abreast of what's available, and affordable, in this rapidly changing area of electronics.

15.9.1 Software

The effective use of microcontrollers requires some investment in software tools (compilers, assemblers, debuggers) that run on some host-computer platform. Along with the cost of the tools comes a significant investment in time to learn how to use them. The great majority of contemporary

programming is done in C/C++, although there is a place for programming at the assembly-language level.

A. C/C++

It's important to realize that there are vendor-specific variations in the C/C++ languages, when coding for a microcontroller: there are libraries that deal with the internal peripherals (SPI port, ADC, timers, and so on), and there are low-level issues such as the configuration of idiosyncratic memory spaces.[47] There are complications connected with timing (clock frequencies, baud rate, timer intervals), and also with processor word width (8/16/32 bits). The bottom line is that the code you are writing is not vanilla C, and consequently is not so easily ported from one microprocessor family to another.[48]

B. Assembly Code

Some microcontroller coding is done in the processor's native assembly code. This is fussy stuff, and it's generally not the right way to go when you are writing a program that is complicated, with complex branching and control, arithmetic functions, and the like. But it lets you get closer to the metal, with access to instructions (such as bit operations, or read–modify–write) that are inaccessible in a compiled language like C. And while it's perfectly OK to write in assembly code for simple microcontroller applications, you've got to remember that such code is difficult to port to a different processor family.

Assembly language coders can write highly optimized loops and timing-critical code (e.g., for a digital signal processor); and assembly-language routines can be invoked from C when needed.[49] Furthermore, a compiled program intended to run in stand-alone mode (i.e., without an operating system) will necessarily acquire some startup assembly code, put there by the development system, that performs tasks such as initializing memory and interrupt vectors, copying executable code from nonvolatile memory to internal RAM (if so instructed), and the like. It's helpful to be able to understand this code, particularly when debugging.

C. BASIC

The BASIC "Stamp" series, made by Parallax, Inc., are tiny (stamp-sized) circuit boards based around PIC or Ubicom (formerly Scenix) microcontroller variants that include a BASIC interpreter in the μC's built-in ROM. They also include a voltage regulator, crystal, and nonvolatile EEPROM for user program and data storage. They come in 14-pin SIP, and in 24-pin- and 40-pin-wide (0.6″) DIP, with prices beginning at $30.[50] These modules include things like digital I/O port pins, PWM, serial port, and I^2C, supported by Parallax's extensions[51] to the BASIC language (called PBASIC); the PBASIC source code is not compiled, but it is stored in a compressed ("tokenized") form, from which it is executed by the on-chip interpreter. Typical BASIC commands occupy 2 to 4 bytes, in their compressed form, when loaded into the Stamp's EEPROM. Because these devices run an embedded *interpreter* (rather than executing compiled code), they are not terribly fast – a few thousand instructions per second – but their simplicity has made them quite popular.

People tend to think of BASIC as an inefficient language (in both speed and memory requirements) – but in fact there are BASIC *compilers* that create good assembly code that runs fast. They don't require a preloaded run-time interpreter program, and they target code for many popular microcontrollers (e.g., AVR, PIC, and ARM).[52]

The BASIC language provides easy access to microcontrollers for the unsophisticated or beginning programmer. Other manufacturers have moved into this niche: you can get, for example, stamp-sized boards with a 32-bit ARM7 that runs a *compiled* BASIC for $50 (ARMexpress, from Coridium).[53]

---

[47] The 8051 architecture, for example, has internal and external code spaces, internal and external data spaces, bit-addressable areas, and special function registers. And, to complicate matters further, modern 8051 variants often have some *internal* "external" memory! The AVR and ARM processors are considerably simpler in this regard; the ARM, in particular, has a single "flat" address space.

[48] There are some pleasant exceptions to this complicated landscape, for example, the *Arduino* platform, which provides simple software and libraries that make programming as easy (or perhaps even *easier*) than the exemplary BASIC Stamp.

[49] But a *cautionary note* on inline assembly code within a C/C++ program: contemporary compilers will "optimize away" program code, assuming they know better than the human programmer; this can create serious havoc.

[50] Well, $29 actually; but we are weary of being *nined* to death, so we routinely round away the trailing 9.

[51] For example, "BUTTON" debounces an input button, performs an auto-repeat, and branches to an address if the button is in a target state.

[52] Examples are GCBASIC (open source), Swordfish, Proton PICBASIC, mikroBasic, and microEngineering Labs PicBasic compiler for the PIC; BASCOM-AVR and the GNU compilers for the AVR; OshonSoft for PIC, AVR and Z80; and several compilers for the ARM family.

[53] Some other BASIC-programmed microcontrollers are BasicX (NetMedia), PICAXE (Revolution Education), KicChip, C Stamp (A-WIT Technologies), CUBLOC (Comfile Technology), and ZBasic (ZBasic/Elba).

D. Java, Python

Some microcontrollers, notably those with ARM cores, include some hardware support for languages like Java. And some interpreted scripting languages have been ported to microcontrollers. These interpreted languages are convenient for high-level jobs that are not time-critical, for example a user interface; but be cautious when counting on them for timing dependent tasks like robotic control. It's often better to devote a separate microcontroller for the latter, with appropriate real-time programming.

15.9.2 Real-time programming constraints

Programming a microcontroller is similar to ordinary computer programming, but with a few important differences. We've already mentioned things like the need to initialize "internal peripherals" through special-function configuration registers, vendor-specific language augmentations, and the fact that run-time code is often just a stand-alone program, without anything like the usual operating system.

An important additional constraint in many applications is the need to conform to critical timing. Some examples are an ADC that must take accurately periodic samples, a serial port (USB, say) whose timing must meet a tight specification, or the generation of analog video.

When you're not dealing with breakneck speeds, the built-in timers offer a good solution: in the first case, you might use one of the μC's internal timers, brought to an output pin, to trigger the ADC at some leisurely rate, say 10 ksps; and the ADC's conversion complete could generate an interrupt, so you can fetch the result. But if you're pushing the speed limits, you may have to resort to carefully coded assembly language, taking account of the microcontroller's execution speed (clocks per instruction). This is fussy stuff, and you have to take care to equalize timings on branches and loops. And the code you generate is not portable, depending as it does on clock speed and processor type.

Timing example: 16-bit serial ADC

To illustrate this with a specific example, consider the LTC1609 200 ksps 16-bit successive-approximation ADC with serial data output, which we saw in Chapter 13 as the back end of a 16-channel data acquisition system (§13.12.1, Figure 13.76). Among other things in systems like this, you need to worry about digital noise contaminating the analog input. To deal with this problem, this converter happily offers several modes: for example, in "internal clock mode" it generates the conversion clocking pulses, and you grab the serialized bits flying your way at exactly the right times; it's too fast for interrupts, so you have to code a highly constrained loop. In another mode, it does the conversion in a burst, while the interface is quiet; then you clock out the multibit data while the converter is idle.

Figure 15.23 shows how it works, in the internal clock mode. You initiate conversion with a CONV′ pulse, after which it gives you the serial data (MSB first, as with all successive-approximation converters) along with a clock. Although the timing is quite relaxed in terms of the speed of standard digital logic hardware (gates and flip-flops), it requires substantial minimum processor speed to handle properly in *software*.

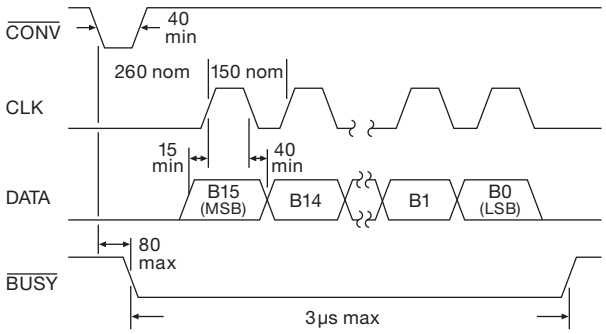


Figure 15.23. Serial data timing of the LTC1609 16-bit ADC when operating in "internal clock mode." Times are in nanoseconds.

Take a look at Pseudocode 15.6. The WAIT-FOR-DATA loop has to complete in 150 ns, which, as we'll see presently, requires at a minimum a processor with a ∼30 MHz clock (with one instruction per clock, including a 16-bit shift instruction). Older PICs, etc., need not apply! Note that we command the channel switching and PGA (programmable gain amplifier) gain setting ("setup next channel") immediately after starting the current conversion, which may seem counterintuitive. But this is the right place to do that, because the PGA needs 2 μs to settle, and the ADC needs another 2 μs to acquire the PGA's output before the subsequent conversion is initiated. And we've got the time to send these setup commands, because there's a ∼200 ns delay after CONV′ before the ADC starts gushing clocks and data.

After EXITing from this loop, you've got ∼ 2 μs to do something with the data, and anything else that needs doing, before the converter is ready for its next conversion at its full speed of 200 ksps (of course, you're always free to run it slower).

Let's look more closely at the critical WAIT-FOR-DATA loop timing, taking it step by step. Our program takes one instruction to check if it's already gotten all the data bits,

Pseudocode 15.6 ADC serial loop pseudocode

```
Channel
    wait until timer
    set CONV′ LOW
    NOP
    set CONV′ HIGH
    setup next channel
Wait-for-data
    if BUSY′ HIGH, Exit
    if CLK LOW, repeat Wait-for-data
    get & shift data bit into 16-bit word
    repeat Wait-for-data
Exit
    store 16-bit word
```

another to see if a new bit is ready (ADC output clock HIGH), one to get it, and one to go back for more. Assuming a 30 MHz single-cycle CPU clock, this loop takes 133 ns. Let's follow what happens as the μC grabs the ADC data clocked every 150 ns. It grabs the first bit someplace in the ADC's 75 ns clock HIGH time, and about 17 ns earlier on each successive grab. After a few bits it's too early, landing in an ADC clock LOW; so it has to loop back, costing it 66 ns and pushing it close to the end of the 75 ns ADC clock HIGH. The μC takes another 33 ns to grab the data bit, approaching the end of the 40 ns ADC data-valid hold time.

As the μC grabs the successive 16 bits, it's all over the place – not a pretty sight. But we're blind and can't see; short of having enough time to add two more instructions and output a marker pulse, there's no way from the outside to see exactly when the processor does what it does. Instead, it's necessary to do a careful thought-process analysis, a common situation with real-time programming and time-sensitive tasks. In this case we'd realize what needs to happen, and we'd calculate the minimum processor clock speed from the worst-case timing situation. That would happen when the ADC output clock is tested just before a rising edge; the loopback and read takes 100 ns to complete, at which time the data bit has only 5 ns of guaranteed life remaining.

So a 30 MHz CPU clock appears marginally fast enough – but it's a close call. And we're in trouble if the ADC's period is shorter than the "typical" value of 150 ns, or if its duty cycle is less than 50%, making its HIGH time 60 ns, say, instead of the 75 ns we assumed. To get a better safety margin we would need a faster processor. Probably better, though, would be to add an outboard D-FF (e.g., an 'LVC1G74) clocked on the rising edge of CLK, to capture each bit and make it available to the μC throughout the full clock period of 150 ns. This would give us an extra 35 ns (plus the flip-flop's delay time of ∼5 ns). If you don't need the processor speed for other reasons, adding a single flip-flop to relax the needed CPU clock frequency is a worthwhile tradeoff.

This example illustrates a common system design issue, namely the tradeoff between software (and processor cycles) and hardware. We could have used a pair of 'LVC595 or 'VHC595 shift registers (see §10.5.3) to clock in the 16 converted ADC bits, which the microcontroller then reads in byte-wise via a parallel port. This would reduce the software burden to fewer than ten instructions every 5 μs. But (apart from the coding task) program execution is free, whereas two chips take space and money; they also hog processor I/O pins. On the other hand, if you don't have much processor time available, or your processor is too slow, adding two chips makes good sense.

Standardized serial protocols

For standardized serial ports like UARTs, I^2C, Ethernet, or USB, the best approach is to use a microcontroller with dedicated internal hardware. A second choice would be an external "bridge" that converts between USB, say, and a standard serial UART (for example the chips made by FTDI). Hardy souls have been known to implement serial ports in assembly language, but it can be a real tour-de-force. A stunning example is Paul Starkjohann's implementation[54] of 1.5 Mbps USB (version 1.1) in assembly language on an Atmel AVR running with a 12 MHz clock, which required precisely eight instructions per serial bit, during which he had to extract the bit stream from NRZI-encoded data, complete with bit stuffing and end-of-packet characters. This was used by Thomas Baier to control a DDS RF generator, part of a complete vector network analyzer.

Quite apart from the challenge of getting the timing right, serial communication via USB involves plenty of software complexity, in the form of drivers and the like. You should not feel bad about using the plain ol' UART, perhaps with a bridge to USB (with drivers provided by the manufacturer).

15.9.3 Hardware

Contemporary microcontrollers use internal nonvolatile (flash-memory) storage for program code, which you load

[54] See `http://www.obdev.at/products/vusb/index.html` and `http://www.obdev.at/articles/implementing-usb-1.1-in-firmware.html`.

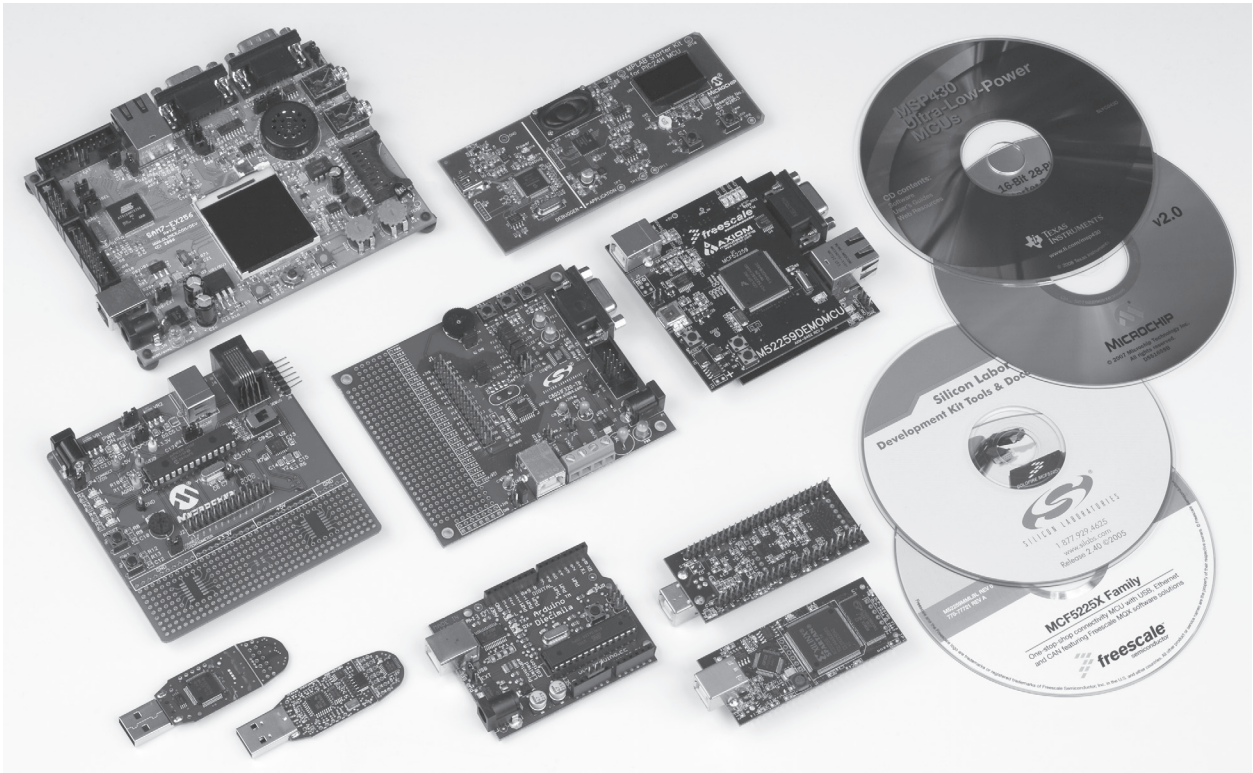


Figure 15.24. Development kits make it easy to get started with a microcontroller family; you get them from the chip manufacturers, or from third-party suppliers. They often include software, cables, and power adaptors. At the top are kits for an Atmel ARM and a Microchip PIC24H; across the middle are PIC24F, Silicon Laboratories C8051F320, and Freescale ColdFire (M52259); across the bottom are Texas Instruments MSP430 (two views), Atmel AVR (ATmega168), and an FPGA devkit (Xilinx Spartan-3E, two views). Suppliers other than the manufacturers are Olimex (ARM), Arduino (AVR), and DLP Design (Spartan).

(while the μC is in-circuit) by one of several methods. You usually do the loading[55] with a commercial "pod" (officially called a "device programmer"), which you buy from the chip manufacturer or third party. If you buy a development kit (Figure 15.24), it will often include a programming pod, along with software (for compiling, simulating, assembling, and loading), and with a circuit board on which there's a microcontroller and other hardware (digital and analog ports, LEDs, a serial port, a programming header, and perhaps some display device). Here are the several loading protocols:[56]

A. UART serial-port bootloader

Some microcontrollers include built-in serial port code (in ROM), so that they wake up listening for programming commands on the UART serial port (a "serial bootloader"). To activate this mode you have to assert one or more pins at reset to signal that you want to program via the UART, and thus put it into program mode. Examples include the Maxim–Dallas DS89C400 series, some Atmel AVRs, and some ARM7 controllers. The communication is via standard serial UART modes (usually 9600 8N1); however, because the microcontroller accepts (unipolarity) logic levels, not bipolarity RS-232, you have to use an interface chip (like a MAX232) between a PC's DE-9 serial-port connector and the target microcontroller.[57] You can, of course,

[55] Which, confusingly, is called "programming" the device. We reluctantly use that term here (interchangeably with "loading," in this hardware context), to mean *physically* programming the microcontroller's flash memory. Do not confuse it with the software programming task of writing the code.

[56] See §§14.5, §14.6.4, and 14.7.4 for more detail on UART, SPI, and JTAG serial data links.

[57] PC serial ports are fast disappearing; you can substitute a USB-to-RS232 adapter, or better, a USB-to-UART chip like the FT232R; the latter matches the logic-level needs of the μC, so no MAX232 style chip is needed.

write your own bootloader code and load it into normal user code memory, so that a microcontroller that permits overwriting the flash code memory can be reprogrammed through the UART. With this method, however, you won't be able to load an unprogrammed chip initially through the UART (the method used by the Arduino project).

B. SPI serial-port bootloader

Some microcontrollers (e.g., the smaller Atmel AVR series) implement a bootloader through the chip's SPI port.[58] Contemporary programming pods connect to the host PC via a USB connection, replacing earlier versions that used the parallel (printer) port, or the serial (RS232) "COM" ports. As with the serial-port bootloaders, you have to assert a pin during boot-up to activate the SPI bootloader. For the AVR controllers, for example, you assert RESET′.

C. JTAG serial-port bootloader

The JTAG serial boundary-scan protocol (§14.7.4), originally intended for test and debugging, is used as a flash-memory bootloader by some microcontrollers. Examples include the higher-pincount AVRs, ARM7, Silicon Labs C8051F series, and Maxim–Dallas MAXQ series. The JTAG port may be one of several choices, as it is with the high-pincount AVRs, which allow bootloading via either JTAG or SPI (as well as parallel programming).

D. Proprietary serial-port bootloader

Some microcontroller manufacturers use their own simple serial protocols, which do not conform to standards like SPI, I^2C, or JTAG. It hardly matters, though, because you generally just use the appropriate pod, as supported by the vendor's development environment. Some of the PIC (Microchip) controllers are programmed this way; and for some of them you must apply a high voltage (+12 V) on a pin called V_{PP} ("high-voltage" programming), while others let you use normal logic-level supply voltages ("low-voltage" programming).

E. USB serial-port bootloader

Microcontrollers that include USB support often implement a USB bootloader option. Examples are some Cypress controllers, and the Atmel ARM and AVR32UC3 series, which can load program memory upon boot from either the USB port, the JTAG port, or the UART.

[58] Which may be one of several bootloader ports on the same μC: the larger Atmel AVR parts, for example, allow bootloader programming via either SPI or JTAG. Additionally, some μCs let you install a custom bootloader in the μC's flash memory, which can bootload from the USB, UART, or Ethernet ports.

F. Parallel loading

Finally, many microcontrollers provide a way to program the internal flash memory via a multiwire parallel connection. In some cases you must apply a higher voltage (e.g., +12 V) to one of the pins; this is sometimes called "high-voltage programming."

G. A "universal" pod?

The "Bus Pirate" project has what appears to be a great tool: for $30 you get a little USB-to-anything piece of hardware, with open-source support. You can open a terminal on your PC and pick from among 1-Wire, I^2C, SPI, JTAG, asynchronous serial (UART), MIDI, PC keyboard, HD44780 LCDs, generic 2- and 3-wire libraries for custom protocols, and scriptable binary bit-bang. You get software support for AVR, JTAG, and some flash-memory in-circuit serial programming. So it's useful for talking to many kinds of chips (not just microcontroller programming) – for example, when debugging the protocols for talking to some new "smart" chip that needs its operating modes set up (we spent plenty of time struggling with a smart alphanumeric LED display strip; this thing would have saved us hours). Bus Pirate version 4 is an upgrade that adds some features (e.g., OpenOCD JTAG mode); it currently costs $40 (`http://dangerousprototypes.com`).

H. Sawing off the branch...

You usually can choose which programming method to use when the μC supports several. However, in controllers where the bootloader can be overwritten, you can clobber the loader, and thus be forced to resort to parallel programming. There are other ways, also, to do what we call "sawing off the branch you're sitting on": we managed to do this with a small AVR chip, when we loaded a program that began by turning off most peripherals, *including the SPI*. That was a big mistake, because it disabled the only port available for serial bootloading. High-voltage parallel programming brought it back to the land of the living.

Another way to terminate your conversation with a microcontroller is to program it to use an external oscillator of the wrong type; that is, one that does not correspond to the actual hardware that is connected (bare crystal, ceramic resonator, or external oscillation signal from an outboard oscillator module). Some microcontrollers protect you from this trap: the Silicon Labs 8051-core processors and the ARM-core processors, for example, boot up in a known state, running from a slow internal oscillator; you have to specifically code any changes from this default. If you do it wrong, just fix the code, and reload from a cold boot.

I. In-circuit debugging

Microcontrollers of recent design include on-chip hardware that lets you debug code during in-circuit execution, by setting breakpoints, examining registers and memory, single-stepping, and so on. (This used to be a big deal, requiring special processor chips outfitted with extra leads, and requiring also considerable outlay of cash.) In-circuit debugging generally uses the same ports as you use for program loading; so you just keep the programming pod attached and iterate a debug–reprogram cycle until it all works right.

Some processor families that currently include these features are the Atmel AVR, processors based on the ARM core, some PIC processors, and some 8051 derivatives (notably those from Silicon Labs).

15.9.4 The Arduino Project

We've mentioned the Arduino Project, approvingly, several times in this chapter. What, exactly, is it?

In the words of the website (www.arduino.cc), "Arduino is an open-source electronics prototyping platform based on flexible, easy-to-use hardware and software. It's intended for artists, designers, hobbyists, and anyone interested in creating interactive objects or environments. . . . The microcontroller on the board is programmed using the [C language with custom libraries] and the Arduino development environment. . . . Arduino projects can be standalone or they can communicate with software running on a computer."

In *our* words, the Arduino *hardware* is a well-designed set of inexpensive boards, based on Atmel microcontrollers (the ATmega AVR series, and the SAM3X ARM Cortex-M series), with all the right external components: USB port (with the FTDI chip to convert to the μC's serial pins), 5 V regulator, SPI port, PWM output, ADC and digital I/O pins, LEDs, and a few additional components. You can buy the standard board (currently called "Uno") assembled, for about $30 from hobbyist suppliers like Adafruit and SparkFun; you can also buy it as a kit or even etch your own (they provide CAD files in Eagle format, or an image file in $*$.png format).

And the *software* development environment is open-source and free: it's a full-blown C, using the GNU C compiler targeted at the AVR and ARM processors. The Arduino software is a simple, user-friendly GUI wrapper around this compiler; it runs on Windows, Mac OS, or Linux, and allows easy code editing and project management. The Arduino firmware (written in C) that runs on the device includes a bootloader that listens for communication over the USB serial link, and either downloads a new user program or times out after a second and runs the program already present in flash (code) memory.

There is a library of nice C functions that you can call from your own programs to do things like text I/O through the USB link, formatting numbers to be printed, setting up timers and interrupts, etc. For example, `val=analogRead(3)` puts the voltage read on analog pin 3 into the variable `val`.[59] Very little of this library code is tied to the Arduino hardware itself, and you can use it in your AVR-based projects. This works well because it's all open source and uses the open-source compiler.

Taken as a whole, the Arduino Project is simply all these components working together. An interesting metric of ease-of-use of any microcontroller hardware-plus-IDE might be the following: how long does it take, after you open the box, before it's doing something interesting? For the Arduino, the answer is "about 20 minutes": download the software, right-click to install the FTDI driver and mini-IDE; then open the Arduino program, type four lines, and off you go! And you don't have to know anything at all about hardware – the little board powers itself through the USB link – nor do you need any extra hardware (except for a standard USB cable).

There are Arduino look-alike products with special features. Many use the same microcontrollers, have the same shield (daughterboard) pinouts, and use the same or similar compilers. For example Linear Technology's "Linduino" board uses their LTM2884 USB isolation chip, providing an isolated USB 2.0 hub or USB peripheral (560 V peak or 2.5 kVrms for one second), complete with an isolated 5 Vdc (to 500 mA). The Linduino board also has LTC's standard DC590 interface board connector, and it's especially well suited for use with their high-resolution ADCs.

15.10 Wrapup

15.10.1 How expensive are the tools?

Both the microcontroller companies, and some third-party manufacturers, make it easy to get started. Many of them offer stripped-down development kits, typically in the range of $50–$200, consisting of (a) a programming pod, (b) a small board with the microcontroller and some trimmings (LEDs, serial port, USB port, analog I/O, and

[59] This ease-of-use comes with some loss of generality, for example in pin assignments.

programming header), and (c) software for compiling, assembling, loading, and debugging.[60]

The compiler that comes with such kits may be a free "evaluation" version of a more capable compiler that you can buy. For example, the inexpensive Silicon Labs kits come with a limited version of the commercial Keil compiler tools. It is full-featured, but limits you to 4 kB of compiled object code, and lacks the floating-point library. The unlimited Keil product lists for ∼$2.5k, from which educational users receive large discounts. Similarly, the Raisonance software (see the 8051 description in §15.3) is available as a free download, again limited to 4 kB of object code, with the unlimited version priced comparably to the Keil product. Another purveyor of high-quality software tools is IAR Systems, with support for most of the popular microcontroller families: 8051, ARM, AVR, Coldfire, MAXQ, PIC, H8, and MSP430, among others. Other suppliers of compilers and debuggers for embedded systems include Green Hills Software, HI-TECH Software, Lauterback, and Rowley Associates.

In a pleasant development, the open-source community has ported the GNU C/C++ compiler to most operating systems, with the ability to target the AVR and ARM family microcontrollers. This free software, plus a programming pod, provides the least costly path to full-featured C-language microcontroller programming. Another encouraging trend is the growing acceptance by semiconductor manufacturers of the need for good software support. Traditionally the chip manufacturers have viewed software as a necessary evil, at best, which they would prefer to have someone else worry about; this seems, happily, to be changing. For example, Atmel integrated the GNU open-source tools into their Windows environment for compiling, simulating, and in-circuit debugging. And IBM's Eclipse open-source development environment has become popular, allowing both manufacturers and users to create plug-ins for specific families of processors. Two examples are Altera's NIOS soft-core μC Micrium, and the open-source ARM tools.

It's important to realize that the programming pods are not generally interchangeable across processor families. Pods aren't very expensive though, so this isn't a serious problem. In some cases the programming hardware allows more than one connection protocol. For example, the Atmel AVR Dragon, which connects to the host via USB and costs about $50, lets you program in SPI, JTAG, parallel, and high-voltage serial modes; it also supports in-system debugging. The simpler AVR ISP pod supports SPI programming only. Programming pods are also sold by third-party and hobbyist suppliers such as Olimex Ltd and Sparkfun Electronics; you can find many more suppliers in the pages of enjoyable hobbyist publications such as *Circuit Cellar*, or *MAKE Magazine*.

[60] If you just want to dip your toe in, there are some demonstrator gadgets that let you program a few LEDs only; these usually plug into a USB port and cost around $10. You can't do much with them, though, and it's better to go with the real devkits.

15.10.2 When to use microcontrollers

Almost always!

Certainly for electronic systems that
(a) have character or graphic displays as part of their user interface;
(b) include chips requiring configuration of internal registers or operating modes;
(c) communicate with a host computer, stand-alone peripherals, network, or wireless devices;
(d) require some computation, storage, format conversion, signal processing, etc.;
(e) require calibration or linearization;
(f) involve sequencing events over time; or
(g) are subject to upgrades or feature revisions.
Microcontrollers should be considered even for traditional "analog" functions such as measurement and control, especially with the growing emphasis on analog-oriented internals in processors from companies such as Analog Devices and Cypress.

Programmable logic devices (PLDs, including FPGAs), by contrast, are generally preferred for tasks requiring critical timing or a high degree of parallelism. They are, however, considerably more difficult than microcontrollers to program and debug. Any system with PLDs will generally include a microcontroller as well; and the latter may take the form of a soft processor core (i.e., configured from the programmable resources in the FPGA) or a hard processor core (i.e., prewired within the "hybrid" FPGA). Examples of FPGA soft cores are the Actel ARM, Altera Nios-II, Lattice Mico, and Xilinx MicroBlaze; examples of prewired processors in hybrid FPGAs include and Altera's ARM, Atmel's FPSLIC with AVR, and Xilinx's PowerPC.

Another way to handle the dual demands of critical timing (e.g., in real-time video or wireless applications) along with the need for the versatility of an embedded microcontroller is to combine an application-specific standard product (ASSP; for example, an MPEG decoder, or cellphone RF subsystem) with a supervisory microcontroller.

15.10.3 How to select a microcontroller

There's a real benefit to using a processor family that the people around you are using, and therefore have available the necessary software and hardware tools, and the experience. Then look at factors like these (depending on your application):
(a) Ports (analog, digital, and communication);
(b) internal functions (e.g., converters, PWM, bare LCD drivers, etc.);
(c) compute speed;
(d) flash, EEPROM, and SRAM memory size;
(e) package configurations;
(f) power dissipation, low-power clock modes, and suspend modes;
(g) software programming, simulation, and in-circuit debugging tools.
The compendium in §15.3 may provide a good starting point.

When choosing a particular part within a microcontroller family, the choices can be overwhelming. It's usually easiest to start with the "premium" part in the series; that is, the one with the fastest clock and most data (RAM) and code memory (flash ROM). It is often the case that most of the family consists of reduced versions of a few premium parts, perhaps with specialized peripherals.

Early microcontrollers were organized around an 8-bit word size, but now there are many 32-bit controllers. Some advantages of the latter (a flat address space, more powerful instructions) are partially offset by considerations like the complexity of startup,[61] somewhat higher prices, and widespread familiarity with the 8-bit processor families.

Even if you code everything in C, most of your troubles will be microcontroller specific: initializing on-board I/Os and peripherals at startup, setting the "fuse bits" (independent of the program code) that control voltage levels and clocking, or dealing with multiple memory pages and idiosyncratic memory locations for peripherals and their configuration. As a result, a simple microcontroller architecture having a flat memory space and standard header files will be much easier to use.

[61] For example, an ARM peripheral needs a lot of code just to power it on, initialize it, connect it to the correct I/O pins, and so on.

The quality and quantity of included libraries can be quite important; for example, the included libraries for TCP/IP, SPI, I^2C, etc., makes it easier to program a Rabbit microcontroller. This is also an important advantage for the Arduino Project, with the large community providing code that has grown around it. Similar comments can be made on the importance of choosing a microcontroller that has a stable compiler–debugger. Similarly, there are a number of advantages with selecting a microcontroller board that is set up to run Linux (such as the Gumstix Overo, or the BeagleBoard). They come with a very stable OS and low-level drivers, they have native preemptive multitasking, they have some nice debugging advantages (one can just use SSH or a serial terminal and get console access), and finally they have impressive hardware (including fairly high-performance graphics controllers).

15.10.4 A parting shot

In case we haven't made it completely evident, microcontrollers are, well, *fun*! Figure 15.25 shows a wacky example, namely Jason Gallicchio's Halloween hat from 2004: it resembles (not coincidentally) an electrified colander, complete with flashing LEDs and a stack of batteries on top. Its notable characteristic, though, is the ability to read the mind of its wearer. The photo shows what happens when the wearer's interlocutor replies to the question "What college do you go to?"

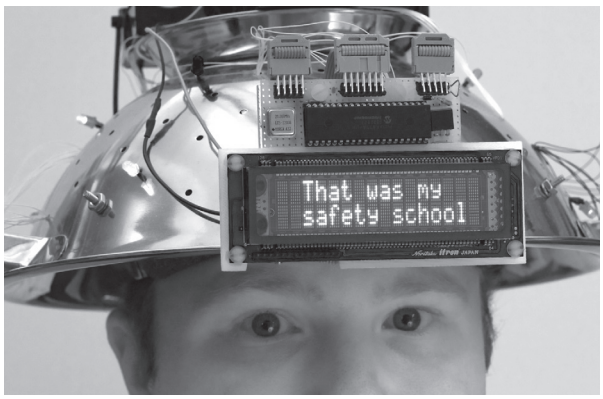


Figure 15.25. "This hat reads my mind."

Review of Chapter 15

The briefest of summaries, of Chapter 15's already brief treatment of microcontrollers.

¶A. Microcontrollers – the 10,000-foot View.
In 1960 the predictions of the future had folks flying around in jetpacks, traveling on supersonic passenger jets, and, most daring of all . . . *a push-button phone in every home!* Put another way, they extrapolated the technologies they knew (transportation, wired telephones), but they missed the Big One (microelectronics, and particularly embedded microcontrollers). We don't have the personal jetpacks, but we've got portable and instant interactive access to news, information, and people. Hey, we don't *need* no stinkin' jetpack!

Microcontrollers (μCs) are standalone processors with a full suite of peripherals integrated on a single inexpensive chip (Figure 15.1). You get substantial computational performance (32-bit processor, 100 MIPs, no problem), plus an assortment of ADCs, DACs, Ethernet, USB, PWM, LCD controller, SPI, multiple UARTs and timers, and of course on-chip program memory and SRAM, all in one chip for a price well below $10; see, for example, the now-vintage example in Figure 10.86, or take a look at NXP's implementations of the ARM Cortex-M4 MCU (microcontroller unit), for example the LPC4088 series. Microcontrollers are useless without their stored *firmware* programs, whose creation can be the major stumbling block to a successful design. So, along with the creation of silicon with stunning integration and performance, the industry has been developing and streamlining the integrated development environment (IDE) – the process of initial code development, simulation, loading onto the target device, and in-circuit debugging.

To the circuit designer, the microcontroller should be thought of as a circuit *component*, like an op-amp and sometimes even less expensive. Even in their simplest varieties they are particularly useful as interfaces between the user and the other circuitry (see for example Figure 15.5); and in their complex varieties they can carry out most of the instrument's functions (as in Figure 15.18). In this chapter we introduced the vast subject of microcontrollers with some illustrative examples, to give a sense of what's possible; we included with each its corresponding *pseudocode*, and for the first example (suntan monitor) we listed the detailed C-language code.

¶B. Popular Microcontroller Families.
In §15.3 we provided an annotated listing of contemporary favorites. The dominant species are the simpler AVR (Atmel) and PIC (Microchip) devices, and the go-to choice for higher performance is the hugely popular ARM-derived processors (licensed by ARM Holdings to more than a dozen semiconductor manufacturers). The latter are used in most of the world's smartphones. The attractive Arduino platform (§15.9.4) includes both AVR- and ARM-based single-board computers (SBCs).

¶C. External Peripherals.
Microcontrollers love to pull the strings of other chips, easily done with a few direct connections (Figure 15.20, §15.8.1), or with simple inter-chip serial buses like SPI (Figure 15.21, §15.8.2) and I^2C (Figure 15.22, §15.8.3). The figures and notes list and describe more than 50 useful peripheral devices.

¶D. Design Hints (Hardware).
The five design examples described in this chapter (suntan monitor, ac power control, frequency synthesizer, thermal controller, and stabilized mechanical platform) included plenty of circuit designs, with corresponding lessons. Here are some of them:
(a) Most μCs include on-chip ADCs, which are attractive when dealing with analog inputs; but don't ignore the simple on-chip comparator, which can sometimes promote both simplicity and better performance (Figure 15.3).
(b) A digital output port bit can happily drive external MOSFETs or BJTs directly (Figure 15.3) or with the assistance of an external gate-driver IC (Figure 15.10); it can also drive a solid-state relay (Figure 15.4).
(c) Don't overlook the simplicity of simple RS-232 serial communication; it's supported in all μCs, and it's alive and well in laptops and PCs running a terminal emulator and connected via a USB or Ethernet adapter (Figure 15.5).
(d) Pushbutton switches can be connected in a matrix arrangement, to minimize wiring (Figure 15.8); they are read with digital port bit polling.
(e) Microcontrollers intended for sensing and control let you connect directly to low-level sensors (Figure 15.11); their PWM output provides an easy way to implement proportional control.
(f) Most μCs include ADCs, making it easy to attach analog-output sensors (e.g., a gyro or accelerometer, Figure 15.18). You can find a zillion little gadgets of this sort at hobbyist sites like sparkfun.com or adafruit.com.
(g) And an easy way to get started is with the Arduino boards and software (§15.9.4); open the box, and you're up and running in 20 minutes.

¶E. Programming Hints (Firmware).
You need microcontroller-specific software tools (compiler, assembler, simulator, debugger) that run on a host

PC; and you need a hardware pod (§15.9.3) to load the object code onto the target μC and to run debugging tools. You also need to be aware of ways in which microcontroller programming differs from ordinary computer programming. Here are some hints:
(a) There are vendor-specific variations in C/C++ having to do with idiosyncrasies of on-chip peripherals (ports, timers, converters, etc.); you're not writing in vanilla C.
(b) The code for a μC must do a significant amount of initialization of modes and internal peripherals; this is fussy stuff, requiring dozens of perfectly configured bytes.
(c) You many need to program in assembly language for time-critical tasks; if so, watch out for too-smart compilers that try to optimize away your code.
(d) Products intended for manufacture should be reviewed and blessed by someone skilled in human interfaces (see §15.2.2E).
(e) Enable the watchdog! Well coded microcontrollers should not crash, but they do.
(f) Keep interrupt routines short; in a simple system you may not need interrupts at all.
(g) Internal timers can generate signals at output pins, very useful when you want to trigger external devices (e.g., an ADC) with constant time intervals (and similarly for on-chip peripherals).

¶F. When to use Microcontrollers.

Almost always! Certainly for electronic systems that (a) have character or graphic displays as part of their user interface; (b) include chips requiring configuration of internal registers or operating modes; (c) communicate with a host computer, standalone peripherals, network, or wireless devices; (d) require some computation, storage, format conversion, signal processing, etc.; (e) require calibration or linearization; (f) involve sequencing events over time; or (g) are subject to upgrades or feature revisions. Microcontrollers should be considered even for traditional "analog" functions such as measurement and control, especially with the growing emphasis on analog-oriented internals in processors from companies such as Analog Devices and Cypress.

Programmable Logic Devices (including FPGAs, Chapter 11), by contrast, are generally preferred for tasks requiring critical timing, or a high degree of parallelism. They are, however, considerably more difficult than microcontrollers to program and debug.

¶G. How to select a Microcontroller.

Look first at a processor family that the people around you are using, and therefore have available the necessary software and hardware tools, and the experience. Then look at factors like these (depending on your application): (a) ports – analog, digital, and communication; (b) internal functions (e.g., converters, PWM, bare LCD drivers, etc.); (c) compute speed; (d) flash, EEPROM, and SRAM memory size; (e) package configurations; (f) power dissipation, low-power clock modes, and suspend modes; (g) software programming, simulation, and in-circuit debugging tools. The compendium in §15.3 may provide a good starting point.

The quality and quantity of included libraries can be quite important (e.g., the Arduino Project, with the large community providing code that has grown around it). Likewise, it's desirable to choose a microcontroller that has a stable compiler–debugger.

When choosing a particular part within a microcontroller family, the choices can be overwhelming. It's usually easiest to start with the "premium" part in the series, that is, the one with the fastest clock and most data (RAM) and code memory (flash ROM). It is often the case that most of the family consists of reduced versions of a few premium parts, perhaps with specialized peripherals.